

CORE 02 — Computer Organization & Architecture

Shared-core file 2 of 3. One study effort, marks in all three electives.

Why this matters

Elective	Where it appears	Weight
Computer Forensic	TP-I §9 — the <i>longest single syllabus entry in the entire CF paper</i> : basic structures and operational concepts, instruction formats, instruction execution and sequencing, addressing modes, stacks, queues, subroutines, register transfers, execution of a complete instruction, RAM/ROM types, cache memories, performance (memory interleaving, hit rate) , memory hierarchy, virtual memory, address translation , secondary memories, I/O organisation, memory-mapped / isolated / linear-select I/O addressing, programmed I/O, interrupt and DMA , synchronous and asynchronous buses, standard interface buses. Also TP-I §4 (computer hardware, memory, storage).	Very high — expect a 15-mark question
CS Degree	Paper-I §4 — "Software and Hardware organization of computers, addressing modes, CPU design, Memory organization, I/O organization, DMA Data transfers"	1 of 6 Paper-I subjects
CS Diploma	Paper-I §3 (storage devices, ROM/PROM/EPROM, registers, cache, RAM, comparative capacity/speed, SDRAM vs DDR, magnetic and optical storage) and §4 (motherboard: system bus, DIMM, AGP, IDE, PCI, serial/parallel ports, USB, BIOS ROM, CMOS) and §1.4 (functional block diagram, CPU/ALU/CU)	High — plus a lot of easy MCQ marks

Strategic note. Notice that the CF and Degree versions want **architecture** (cache mapping, DMA, addressing modes, virtual memory) while the Diploma version wants **concrete hardware facts** (what a DIMM slot is, EPROM vs EEPROM, SDRAM vs DDR). This file covers both. The Diploma hardware facts are almost free MCQ marks — do not skip Part G.

The three highest-yield sub-topics, in order:

1. **Cache memory** (mapping + numericals) — appears explicitly in the CF syllabus and is a guaranteed 10/15-mark question.
2. **Addressing modes** — a standard 5- or 10-marker in every organisation paper ever set.
3. **DMA and I/O transfer methods** — named explicitly in both CF and Degree syllabi.

Topic checklist

- ☐ 1. Von Neumann model, stored-program concept, functional units, Von Neumann bottleneck
- ☐ 2. Von Neumann vs Harvard architecture
- ☐ 3. Bus structures (single, two, three bus); data/address/control bus
- ☐ 4. CPU registers: PC, IR, MAR, MDR, AC, SP, general-purpose, status/flags
- ☐ 5. Instruction formats: 0/1/2/3-address; instruction length trade-offs
- ☐ 6. Instruction cycle: fetch—decode—execute, with interrupt cycle
- ☐ 7. Register Transfer Language (RTL) / micro-operations
- ☐ 8. Addressing modes — all eleven, each with an example
- ☐ 9. Stacks, queues, subroutine call/return mechanism
- ☐ 10. CPU datapath design; single-bus and multi-bus organisation
- ☐ 11. Control unit: hardwired vs microprogrammed; horizontal vs vertical microcode
- ☐ 12. RISC vs CISC
- ☐ 13. Memory hierarchy; locality of reference
- ☐ 14. RAM types (SRAM, DRAM, SDRAM, DDR1–5) and ROM types (PROM, EPROM, EEPROM, Flash)

- [] 15. Memory chip organisation and address decoding; chip-count problems
- [] 16. **Cache: direct, associative, set-associative mapping** — address partitioning
- [] 17. **Cache replacement policies** (FIFO, LRU, LFU, Random)
- [] 18. Write policies: write-through, write-back, write-allocate
- [] 19. **Hit ratio and average access time numericals** (single and multi-level)
- [] 20. **Memory interleaving** (low-order and high-order)
- [] 21. **Virtual memory, paging, page table, TLB, effective access time**
- [] 22. Segmentation and segmented paging
- [] 23. Secondary storage: hard disk geometry, capacity and access-time calculations
- [] 24. Optical and solid-state storage
- [] 25. I/O organisation: memory-mapped vs isolated I/O; linear selection
- [] 26. **Programmed I/O, interrupt-driven I/O, DMA** — full comparison
- [] 27. Interrupts: types, vectored/non-vectored, daisy chaining, priority
- [] 28. DMA modes: burst, cycle stealing, transparent
- [] 29. Synchronous vs asynchronous buses; handshaking
- [] 30. Standard interface buses: ISA, PCI, PCIe, AGP, USB, SATA, SCSI
- [] 31. **Pipelining**: speedup, efficiency, throughput, and the three hazard classes
- [] 32. Motherboard components (Diploma-specific)

PART A — THE VON NEUMANN MODEL AND FUNCTIONAL UNITS

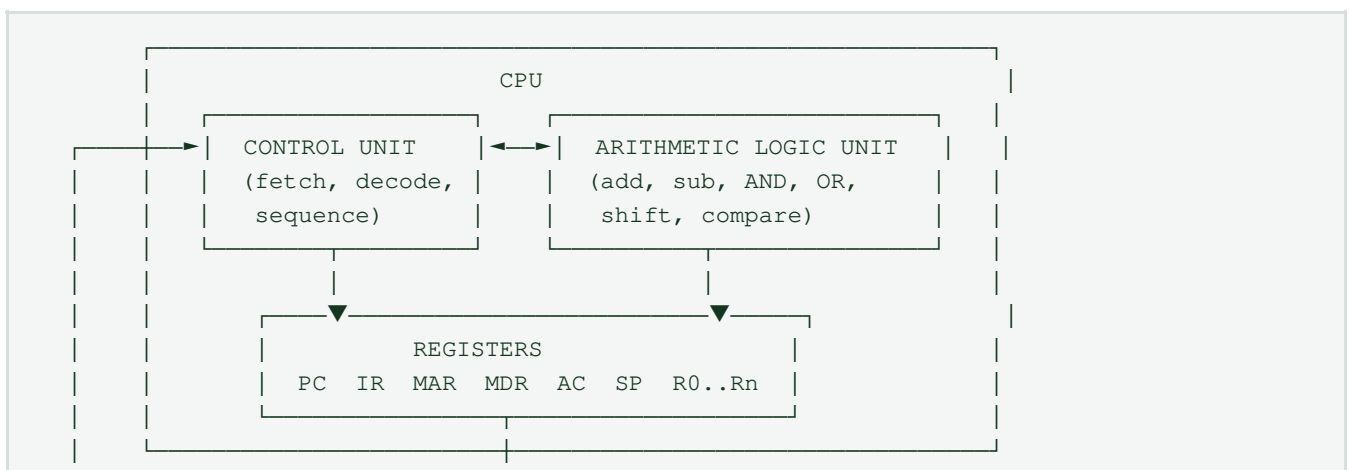
A1. The stored-program concept

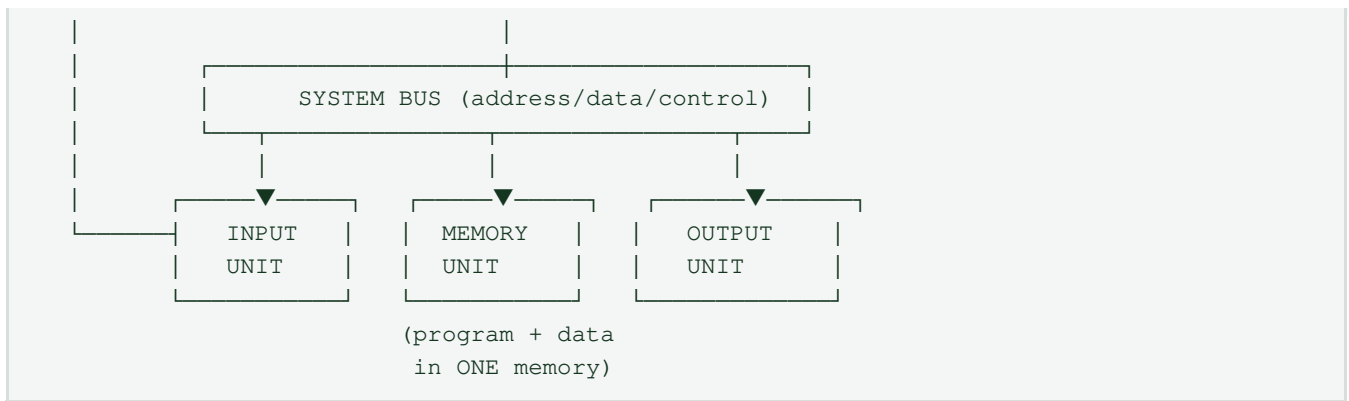
Concept — plain English first

Before 1945, "programming" a computer like the ENIAC meant physically rewiring it — days of plugging cables to change what it computed. **John von Neumann's insight (with Eckert and Mauchly) was startlingly simple: instructions are just numbers, so store them in the SAME memory as the data.** Then changing the program is only a matter of loading different numbers into memory. That single idea created the software industry.

The consequence is the structure every general-purpose computer still has: a memory holding both program and data, a processing unit that reads instructions from it one at a time, and a control unit that sequences the whole thing.

The five functional units





Unit	Function
Input unit	Accepts data and programs from the outside world (keyboard, mouse, scanner, disk) and converts them to machine-readable form
Memory unit	Stores the program and data. Primary (RAM/ROM/cache) and secondary
ALU	Performs all arithmetic (+, −, ×, ÷) and logic (AND, OR, NOT, XOR, shift, compare) operations
Control unit	Fetches, decodes and sequences instructions; generates timing and control signals for every other unit. The "nerve centre" — it does no processing itself.
Output unit	Presents results (monitor, printer, plotter, speaker)

CPU = ALU + Control Unit + Registers.

"**Central Processing Unit**" is CU + ALU + register set; the memory and I/O are outside it.

Key points

1. **Stored-program concept:** instructions and data share one memory, in the same binary format, and are indistinguishable except by how they are used.
2. **Sequential execution:** instructions execute one after another unless a branch changes the flow. The **Program Counter** holds the address of the next instruction.
3. **The Von Neumann bottleneck:** because there is a *single* bus between CPU and memory, instruction fetches and data accesses must take turns. The CPU can process faster than the bus can supply, so the CPU stalls. **This is the single most important limitation of the model** and the reason caches, pipelining, and Harvard-style split L1 caches exist.

Von Neumann vs Harvard architecture

	Von Neumann	Harvard
Memories	One memory for instructions and data	Separate instruction and data memories
Buses	One shared bus	Two independent buses
Simultaneous fetch of instruction + data	No	Yes
Bottleneck	Yes (the classic bottleneck)	Largely avoided
Cost / complexity	Lower	Higher
Flexibility	Memory freely allocated between code and data	Fixed split

Typical use	General-purpose computers, PCs	DSPs, microcontrollers (PIC, AVR, ARM Cortex-M)
Modified Harvard	Modern CPUs: unified main memory but split L1 instruction and data caches — Von Neumann outside, Harvard inside	

Bus structures

Bus	Carries	Direction	Width determines
Address bus	The memory or I/O address	Unidirectional (CPU → memory)	Addressable memory size = 2^{width} . 16-bit → 64 KB; 20-bit → 1 MB; 32-bit → 4 GB
Data bus	The actual data word	Bidirectional	Word size / transfer size per cycle
Control bus	READ, WRITE, MEMR, MEMW, IOR, IOW, clock, reset, interrupt, bus request/grant	Mixed	—

Single-bus organisation: everything on one bus. Cheap, but only one transfer at a time — slow. **Two-bus / three-bus organisation:** more parallel paths inside the CPU, so a micro-operation like $R3 \leftarrow R1 + R2$ can complete in one clock instead of three.

CPU registers — know every one

Register	Full name	Function
PC	Program Counter (a.k.a. Instruction Pointer)	Holds the address of the NEXT instruction . Incremented during fetch.
IR	Instruction Register	Holds the instruction currently being executed/decoded
MAR	Memory Address Register	Holds the address of the memory location to be accessed. Connects to the address bus. Its size = address bus width.
MDR / MBR	Memory Data (Buffer) Register	Holds the data being read from or written to memory. Connects to the data bus. Its size = data bus width.
AC	Accumulator	Holds one ALU operand and the result (in accumulator-based machines)
SP	Stack Pointer	Holds the address of the top of the stack
Flags / PSW	Status Register / Program Status Word	Condition codes: Z (zero), C (carry), S/N (sign/negative), V/O (overflow), P (parity), plus interrupt-enable and mode bits
General-purpose	R0...Rn	Programmer-visible working registers
TR	Temporary Register	Internal scratch, invisible to the programmer
Index / Base registers	—	Used in indexed and base-relative addressing

A very common MCQ: "Which register is not accessible to the programmer?" — **MAR and MDR** (and TR). PC and SP are visible on most architectures.

Likely exam questions — Von Neumann model

- If the opcode field is n bits, at most **2^n distinct instructions** are possible.
- **Expanding opcode** technique: use short opcodes for common 3-address instructions and longer opcodes for rarer 0- and 1-address instructions, so the total instruction length stays fixed.

The four instruction classes — worked comparison

Task: evaluate $X = (A + B) \times (C + D)$

Format	Program	Instructions	Comment
3-address	<pre>ADD R1, A, B ADD R2, C, D MUL X, R1, R2</pre>	3	Shortest program; longest instructions
2-address	<pre>MOV R1, A ADD R1, B MOV R2, C ADD R2, D MUL R1, R2 MOV X, R1</pre>	6	One operand is both source and destination (destructive)
1-address (accumulator)	<pre>LOAD A ADD B STORE T LOAD C ADD D MUL T STORE X</pre>	7	AC is implicit; needs a temporary location T
0-address (stack)	<pre>PUSH A PUSH B ADD PUSH C PUSH D ADD MUL POP X</pre>	8	Operands implicit (top of stack). Requires the expression in postfix/RPN: $AB+CD+\times$

The trade-off, stated for the exam:

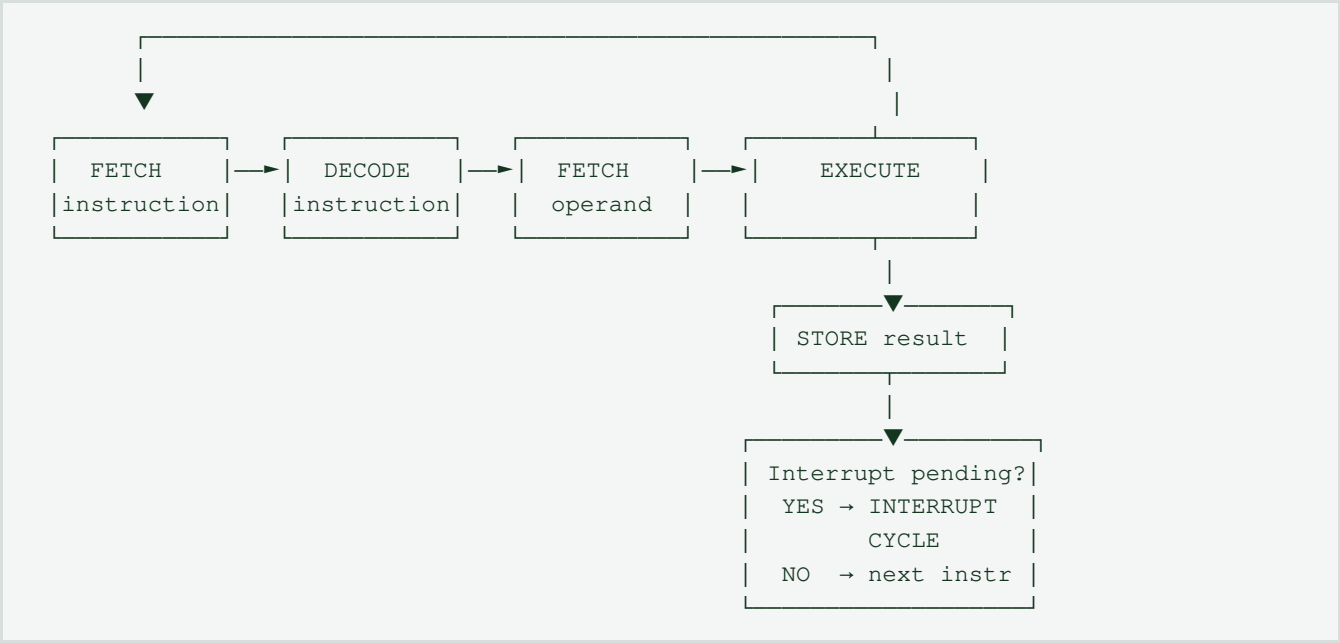
	Program length	Instruction length	Memory traffic per instruction	Hardware
3-address	Shortest	Longest	Most	Most registers needed
2-address	Short	Medium	Medium	Common (x86)
1-address	Long	Short	Low	Simple (early micros, 8085)
0-address	Longest	Shortest	Lowest per instruction	Simplest decode; needs a stack

B2. The instruction cycle

Concept

The CPU is, at bottom, an infinite loop: **get the next instruction, work out what it is, do it, repeat**. Everything else — pipelining, interrupts, caches — is an optimisation or an interruption of this loop.

The cycle in detail



The fetch cycle in RTL — memorise this three-line sequence

```
T0:  MAR ← PC           (put the next address on the address bus)
T1:  MDR ← M[MAR], PC ← PC + 1 (read memory; simultaneously advance the PC)
T2:  IR ← MDR           (move the instruction into the instruction register)
```

Note **PC ← PC + 1 happens during the fetch, not after execution** — a classic MCQ. (On byte-addressed machines it's PC ← PC + instruction length.)

Worked example 1 — full execution of **ADD R1, (R3)** in RTL

Assume a single-bus CPU, memory-referencing add, register-indirect operand.

Step	Micro-operations	Comment
T0	MAR ← PC	fetch begins
T1	MDR ← M[MAR] ; PC ← PC + 1	read instruction, bump PC
T2	IR ← MDR	instruction now in IR
T3	(decode IR)	CU works out: ADD, reg-indirect
T4	MAR ← R3	operand ADDRESS comes from R3
T5	MDR ← M[MAR]	fetch the operand
T6	R1 ← R1 + MDR ; set flags	execute in the ALU

Seven steps, three memory accesses avoided by having R3 already hold the address. Notice that **the number of memory references depends entirely on the addressing mode** — which motivates the next section.

The interrupt cycle

At the end of every instruction the CPU checks for a pending interrupt. If one exists:

1. Complete the current instruction (interrupts are checked between instructions)
2. Push PC and PSW (flags) onto the stack [save the return context]
3. Disable further interrupts (or mask lower priorities)
4. Load PC with the address of the Interrupt Service Routine (from the vector table)
5. Execute the ISR
6. IRET: pop PSW and PC → resume the interrupted program exactly where it left off

B3. Addressing modes — the complete set

Concept

The operand field of an instruction is only a handful of bits. Addressing modes are the different **rules for turning those bits into the actual location of the data**. They exist to give the programmer three things: **shorter instructions** (you can point at a big address with a small field), **flexibility** (arrays, pointers, recursion), and **relocatability** (code that works wherever it's loaded).

The one question to ask of every mode: **"How many memory accesses does it take to get the operand?"** That is the axis examiners test.

The eleven modes

#	Mode	Effective Address (EA)	Example	Operand is	Memory refs for operand
1	Implied / Implicit	none — operand is implicit in the opcode	CLC, CMA, RTS, PUSH	Implicit (AC, flags, stack top)	0
2	Immediate	none — the operand IS in the instruction	MOV R1, #25	The value 25 itself	0 (came with the instruction)
3	Register (direct)	EA = register name	MOV R1, R2	Contents of R2	0
4	Register indirect	EA = contents of a register	MOV R1, (R2)	M[R2]	1
5	Direct / Absolute	EA = the address field	LOAD 5000	M[5000]	1
6	Indirect	EA = M[address field]	LOAD @5000	M[M[5000]]	2
7	Indexed	EA = address field + index register	LOAD 2000(R1)	M[2000 + R1]	1
8	Base register	EA = base register + displacement	LOAD R1, 40(BX)	M[BX + 40]	1
9	Relative (PC-relative)	EA = PC + offset	BEQ +12	M[PC + 12]	1
10	Autoincrement	EA = (R); then $R \leftarrow R + d$	LOAD (R1) +	M[R1], then R1 advances	1
11	Autodecrement	$R \leftarrow R - d$; then EA = (R)	LOAD -(R1)	R1 decremented first, then M[R1]	1
—	Stack	EA = top of stack (via SP)	PUSH A, POP B	M[SP]	1

Worked example 2 — one instruction, every mode

Setup: PC = 200, R1 = 400, address field of the instruction = 500.

Memory contents: M[400] = 700, M[500] = 800, M[700] = 900, M[800] = 1000, M[900] = 1100.

Mode	EA computed as	EA	Operand fetched
Immediate	—	—	500 (the address field is the data)
Direct	500	500	M[500] = 800
Indirect	M[500]	800	M[800] = 1000
Register	R1	—	400 (contents of R1)
Register indirect	R1	400	M[400] = 700
Indexed (index = R1)	500 + 400	900	M[900] = 1100
Relative (PC already incremented to 201, say)	201 + 500	701	M[701]
Autoincrement (R1)	400, then R1 ← 401	400	M[400] = 700
Autodecrement (R1)	R1 ← 399, then 399	399	M[399]

Draw this exact table in the exam — it is the standard way this question is set and it demonstrates every mode at once.

Why each mode exists — the "application" marks

Mode	Why it exists / typical use
Immediate	Loading constants; no memory access → fastest
Direct	Simple global variables
Indirect	Pointers ; lets an instruction with a small address field reach anywhere
Register	Fastest operand access of all — registers are inside the CPU
Register indirect	Pointers held in registers; walking data structures
Indexed	Array element access — base of the array in the address field, subscript in the index register. Incrementing the index steps through the array.
Base register	Program relocation and segmentation — the base register is reloaded when the program moves
Relative	Position-independent code : branch instructions (a branch is nearly always to a nearby address, so a short offset suffices)
Autoincrement/ decrement	Stepping through arrays and stacks without an explicit increment instruction; the basis of <code>*p++</code> in C
Stack	Subroutine calls, expression evaluation, parameter passing

Stacks, queues and subroutines (explicitly in the CF syllabus)

Stack: LIFO. A stack pointer (SP) marks the top. **PUSH**: $SP \leftarrow SP - 1$; $M[SP] \leftarrow \text{data}$ (for a stack growing downward). **POP**: $\text{data} \leftarrow M[SP]$; $SP \leftarrow SP + 1$.

Queue: FIFO. Two pointers, FRONT and REAR. Insert at rear, delete at front. Used for I/O buffers and scheduling queues.

Subroutine call/return mechanism — the classic 5-marker:

```
CALL:  push the return address (current PC) onto the stack
       PC ← subroutine start address
RETURN: pop the return address from the stack into PC
```

- The **stack** (not a fixed register) is used for the return address precisely because it allows **nested and recursive** calls — each level gets its own saved address.
- The block pushed for one call — return address, saved registers, parameters, local variables — is called a **stack frame** or **activation record**.
- Parameter passing methods: in registers (fastest, limited), in memory, or **on the stack** (most general, supports recursion).

Reverse Polish (postfix) evaluation on a stack — worked:

Infix $(A + B) \times (C + D) \rightarrow$ postfix $A B + C D + \times$

Read A → push A	Stack: A
Read B → push B	Stack: A B
Read + → pop B, pop A, push A+B	Stack: (A+B)
Read C → push C	Stack: (A+B) C
Read D → push D	Stack: (A+B) C D
Read + → pop, pop, push C+D	Stack: (A+B) (C+D)
Read × → pop, pop, push product	Stack: (A+B) × (C+D) ← the answer

Likely exam questions — Instructions and addressing

Q	Marks
Explain immediate, direct, indirect and register indirect addressing with examples.	5
What is the instruction cycle? Write the fetch cycle in register transfer notation.	5
Explain how subroutine call and return are implemented using a stack. Why is a stack necessary for recursion?	5
Compare 0-, 1-, 2- and 3-address instruction formats by writing the code for $X = (A+B) \times (C-D)$ in each.	10
Explain all the addressing modes of a computer. Given PC=200, R1=400, address field=500 and the memory contents shown, compute the effective address and operand for each mode.	15
Describe the instruction cycle in detail, including the interrupt cycle. Write out the complete register-transfer sequence for the execution of a memory-reference ADD instruction and explain the role of PC, IR, MAR and MDR at each step.	15

MCQ traps — Instructions and addressing

Trap	Truth
Immediate mode requires one memory access	Zero — the operand arrives with the instruction.
Indirect addressing needs one memory access	Two (one for the address, one for the data).
Indexed and base-register addressing are identical	Mechanically similar, but indexed varies the index for array traversal while base varies the base for relocation.

Relative addressing uses the accumulator	It uses the PC .
Stack machine uses infix notation	Postfix (RPN) .
PC is incremented after execution	During the fetch cycle.
3-address code gives the shortest instructions	It gives the shortest program but the longest instructions .
An n-bit opcode allows n instructions	2^n instructions.

PART C — CPU DESIGN AND THE CONTROL UNIT

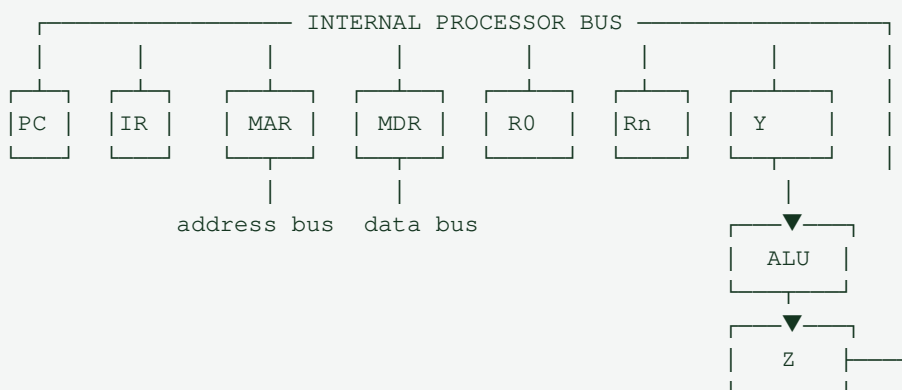
C1. CPU datapath

Concept

The **datapath** is the collection of registers, ALU and buses through which data actually flows. The **control unit** is what opens and closes the gates along that path at the right moments. Together they execute instructions.

The key design variable is **how many internal buses** you provide. With a single internal bus, only one register can drive the bus at a time, so an ALU operation needs three separate steps (load operand 1 into a temp, put operand 2 on the bus, write the result back). With three buses, all of it happens in one clock.

SINGLE-BUS CPU ORGANISATION (hand-draw this)



Y and Z are temporary registers that hold ALU inputs/outputs because only ONE value can be on the single bus at a time.

Micro-operation $R3 \leftarrow R1 + R2$ on a single-bus CPU takes three clocks:

```

T1:  Y ← R1           (R1 out on bus, Y in)
T2:  Z ← Y + R2       (R2 out on bus, ALU adds Y + bus, result to Z)
T3:  R3 ← Z           (Z out on bus, R3 in)

```

On a **three-bus** CPU it takes **one** clock, because two source buses and one destination bus operate simultaneously.

C2. Control unit: hardwired vs microprogrammed

Concept

The control unit's job is to emit, at every clock tick, the right set of control signals (`PC_out`, `MAR_in`, `Read`, `ALU_add`, ...) for whatever instruction is in the IR. There are two ways to build the thing that does this:

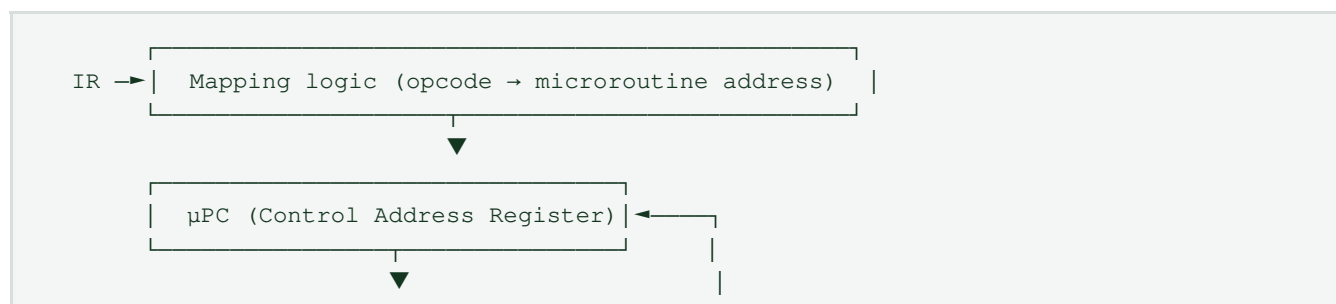
- **Hardwired:** build a fixed combinational/sequential circuit — a state machine of gates and flip-flops — whose outputs *are* the control signals. Fast (it's just logic), but if you want to add an instruction you have to **redesign the hardware**.
- **Microprogrammed (Wilkes, 1951):** store the control signals as **words in a small fast ROM** (the *control memory*). Each word is a **microinstruction** whose bits directly drive the control lines. Executing a machine instruction means running a little program — a **microprogram** — out of that ROM. Slower (an extra memory read per step), but changing the instruction set is now just **rewriting the ROM contents**.

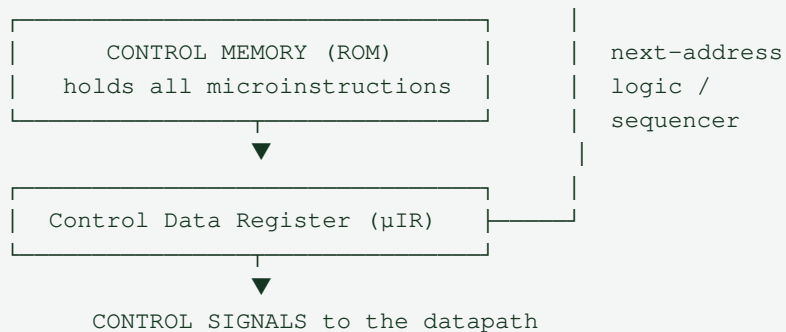
CISC machines (with hundreds of complex instructions) are almost always microprogrammed; RISC machines (few, simple, regular instructions) are hardwired.

Comparison — a guaranteed exam table

Aspect	Hardwired control	Microprogrammed control
Implementation	Fixed logic: gates, decoders, counters, flip-flops (a sequential circuit)	Control memory (ROM) holding microinstructions
Speed	Fast (pure logic delay)	Slower (control-memory read per micro-step)
Flexibility / modification	Very difficult — requires redesigning and refabricating the circuit	Easy — rewrite the microprogram
Design cost / time	High for complex instruction sets	Lower and more systematic
Ability to handle complex instructions	Poor — logic explodes	Good
Chip area	Less (for simple sets)	More (control memory)
Error correction / adding instructions	Hard	Easy
Occurrence	RISC processors	CISC processors
Also called	Random logic control	Firmware control / stored-logic control

Microprogrammed control unit structure





Horizontal vs vertical microprogramming

	Horizontal	Vertical
Encoding	Unencoded — one bit per control signal	Encoded — signals grouped into fields that must be decoded
Microinstruction width	Very wide (hundreds of bits)	Narrow
Parallelism	High — many operations per microinstruction	Low — usually one operation per microinstruction
Control memory size	Large (wide words)	Small (narrow words) but more words needed
Speed	Faster (no decoding step)	Slower (decoder delay)
Microprogram length	Short	Long

Nanoprogramming is a two-level scheme: a vertical microprogram indexes into a horizontal nanoprogram, getting the size advantage of vertical with the parallelism of horizontal.

RISC vs CISC

	CISC	RISC
Instruction count	Large (hundreds)	Small (typically <100)
Instruction length	Variable	Fixed (e.g. 32 bits) — simplifies pipelining
Addressing modes	Many (12–24)	Few (3–5)
Memory access	Almost any instruction can access memory	Only LOAD and STORE ("load-store architecture")
Registers	Few	Many (32+)
Control unit	Microprogrammed	Hardwired
Cycles per instruction	Variable, multi-cycle	Mostly 1 cycle (pipelined)
Pipelining	Difficult	Easy and deep
Code size	Smaller	Larger
Compiler complexity	Simpler compiler, complex hardware	Complex compiler , simple hardware

Locality of reference — the three kinds

Type	Meaning	Everyday cause
Temporal locality	A location referenced now is likely to be referenced again soon	Loop counters, frequently-called functions, stack variables
Spatial locality	Locations near a referenced location are likely to be referenced soon	Sequential instruction fetch, array traversal, struct field access
Sequential locality	A special case of spatial: addresses accessed in increasing order	Instruction streams, file reads

Spatial locality is why caches fetch a whole **block/line** rather than a single word.

Temporal locality is why we keep blocks around instead of discarding them immediately.

The hierarchy

▲ FASTER, SMALLER, COSTLIER PER BIT			
REGISTERS	~1 KB	<1 ns	inside CPU
L1 CACHE (SRAM)	32-64 KB	1-2 ns	split I/D
L2 CACHE (SRAM)	256KB-1MB	~5 ns	per core
L3 CACHE (SRAM)	8-32 MB	~20 ns	shared
MAIN MEMORY (DRAM)	8-64 GB	~60-100ns	
SSD / FLASH	0.5-4 TB	~50-100µs	
MAGNETIC DISK	1-20 TB	~5-10 ms	
MAGNETIC TAPE / OPTICAL / CLOUD	~PB	seconds	offline archive
▼ SLOWER, LARGER, CHEAPER PER BIT			

The ratios to quote: registers are roughly **100×** faster than DRAM; DRAM is roughly **100,000×** faster than disk. That million-fold gap between RAM and disk is why a page fault is catastrophic and a cache miss is merely annoying.

Level	Managed by	Transfer unit
Registers ↔ Cache	Compiler / hardware	Word
Cache ↔ Main memory	Hardware (cache controller)	Block / line (typically 32-64 bytes)
Main memory ↔ Disk	Operating system (virtual memory)	Page (typically 4 KB)
Disk ↔ Tape/archive	User / OS	File

D2. RAM and ROM types

RAM: SRAM vs DRAM

	SRAM (Static RAM)	DRAM (Dynamic RAM)
Storage element	Flip-flop (6 transistors)	Capacitor + 1 transistor
Refresh needed	No	Yes — the capacitor leaks; must be refreshed every few milliseconds
Speed	Fast (~1–10 ns)	Slower (~50–70 ns)
Density	Low (big cell)	High (tiny cell)
Cost per bit	High	Low
Power	Higher static, lower during access	Lower static, but refresh consumes power
Volatile?	Yes (both are volatile)	Yes
Used for	Cache memory, registers	Main memory

The single most-asked RAM MCQ: which needs refreshing? → DRAM, because it stores charge on a capacitor that leaks.

DRAM generations (Diploma syllabus names SDRAM vs DDR explicitly)

Type	Full name	Key property
FPM / EDO DRAM	Fast Page Mode / Extended Data Out	Legacy asynchronous DRAM
SDRAM	Synchronous DRAM	Synchronised to the system clock; transfers once per clock cycle (on the rising edge only)
DDR SDRAM	Double Data Rate	Transfers on BOTH rising and falling clock edges → 2× the bandwidth at the same clock
DDR2		4 transfers per clock at the I/O bus (2× DDR), lower voltage (1.8 V)
DDR3		8 transfers per clock, 1.5 V
DDR4		16 transfers per clock, 1.2 V, higher density
DDR5		Higher speed again, on-die ECC, 1.1 V
RDRAM	Rambus DRAM	Proprietary high-speed, largely obsolete

SDRAM vs DDR — the exam answer in one line: SDRAM transfers data once per clock cycle (rising edge only); DDR transfers on both edges, doubling the data rate without increasing the clock frequency.

Also know: **DIMM** = Dual In-line Memory Module (independent contacts on both sides, 64-bit data path — the modern standard). **SIMM** = Single In-line (contacts on both sides are electrically the same, 32-bit). **SO-DIMM** = the small laptop version.

ROM family — memorise this table, it is pure Diploma MCQ material

Type	Full name	How written	How erased	Reusable?
ROM (Mask ROM)	Read Only Memory	At the factory , during fabrication (masked in)	Cannot be erased	No
PROM	Programmable ROM	Once by the user , using a PROM programmer that blows fusible links	Cannot be erased	No — one-time programmable (OTP)
EPROM	Erasable PROM	Electrically, by a programmer	Ultraviolet light through a quartz window (~20 min), erases the whole chip	Yes, but must be removed from the circuit
EEPROM	Electrically Erasable PROM	Electrically	Electrically, byte by byte , in-circuit	Yes
Flash	Flash EEPROM	Electrically	Electrically, in BLOCKS (faster than EEPROM's byte-wise erase)	Yes — the basis of SSDs, USB drives, BIOS

- All ROM types are **non-volatile** (retain data without power) and **random access**.
- Uses: **BIOS/firmware**, bootstrap loader, lookup tables, embedded programs.
- **NOR flash** allows random read (used for code/BIOS); **NAND flash** is denser and read in pages (used for SSDs and memory cards).

Memory chip organisation and address decoding

A memory chip described as "**1K × 8**" means **1024 locations of 8 bits each**:

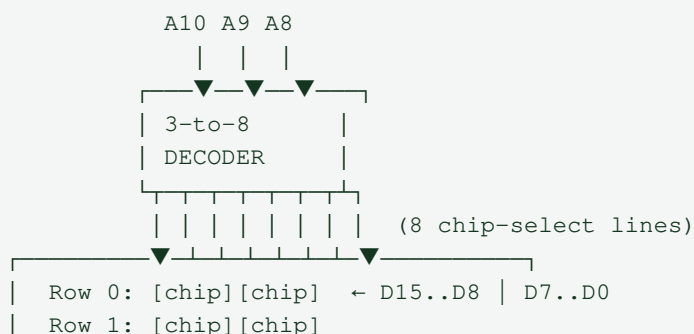
- Address lines needed: $2^{10} = 1024 \rightarrow$ **10 address lines**
- Data lines: **8**
- Total capacity: 1 KB

Worked example 3 — chip-count problem (a very common numerical).

How many 256×8 RAM chips are needed to build a $2K \times 16$ memory system? How many address lines does the system need and how many are decoded?

```
Chips needed vertically (capacity): 2048 / 256 = 8 rows
Chips needed horizontally (width): 16 bits / 8 bits = 2 columns
TOTAL CHIPS = 8 × 2 = 16
```

```
Each chip has 256 locations → 28 → 8 address lines go to every chip (A0-A7)
The system has 2048 locations → 211 → 11 address lines total (A0-A10)
The remaining 3 lines (A8, A9, A10) select WHICH ROW of chips →
    decoded by a 3-to-8 decoder driving the 8 chip-select (CS) lines
```



```

| ... (8 rows × 2 columns = 16 chips)
| Row 7: [chip][chip]
|_____
|
| A0-A7 go to ALL chips in parallel

```

Linear selection (linear decoding): instead of a decoder, use one high-order address line directly as each chip's CS. Cheap (no decoder) but **wasteful of address space** and you must never assert two CS lines at once. **Full decoding** uses a decoder and utilises the entire address space — this is the "linear selection technique of I/O addressing" mentioned in the CF syllabus.

D3. Cache memory — THE key topic

Concept

The CPU is far faster than DRAM. Every time it waits for main memory it wastes hundreds of cycles doing nothing. **Cache is a small, very fast SRAM buffer that sits between the CPU and main memory and holds copies of the memory blocks currently in use.**

The mechanism: when the CPU asks for an address, the cache is checked first. If the data is there — a **hit** — it is delivered at cache speed. If not — a **miss** — the whole **block** containing that address is copied from main memory into the cache (bringing its neighbours along, exploiting spatial locality) and then delivered. Because of locality, hit rates of 95–99% are routine, so the *average* access time is close to the cache's speed even though the *capacity* is main memory's.

The entire design problem is: **where in the cache may a given main-memory block be placed, and how do we know quickly whether it is there?** The three answers are the three mapping techniques.

Terminology

Term	Meaning
Block / Line	The unit of transfer between main memory and cache (e.g. 16 bytes)
Line/Slot	A place in the cache that can hold one block
Tag	Extra bits stored with each cache line identifying which main-memory block it holds
Valid bit	Says whether the line currently holds real data
Dirty bit	Says the line has been modified and must be written back
Hit ratio (h)	hits / total accesses
Miss ratio	1 – h
Miss penalty	The extra time to service a miss

The three mapping techniques

1. Direct mapping

Concept: each main-memory block has **exactly one** cache line it is allowed to occupy, determined by `block number MOD number of cache lines`. Dead simple and dead fast to check

— you go straight to the one possible line and compare one tag. The cost: if a program alternates between two blocks that map to the same line, they evict each other on every access — **thrashing / conflict misses** — even if the rest of the cache is empty.

Address split:

TAG	LINE (index)	WORD/OFFSET
-----	--------------	-------------

2. Fully associative mapping

Concept: a block may go in **any** cache line. Maximum flexibility, so no conflict misses at all — a block is only evicted when the whole cache is full. The cost: to find out whether a block is present you must compare its tag against **every** line's tag. That needs expensive parallel comparison hardware (**content-addressable memory**), which is why fully associative caches are only used when they are small (e.g. TLBs).

Address split:

TAG	WORD/OFFSET
-----	-------------

3. Set-associative mapping — the practical compromise

Concept: divide the cache into **sets** of k lines each ("k-way set associative"). A block maps to exactly one **set** (like direct mapping) but may sit in **any line within that set** (like associative). You only compare k tags, which is cheap, and you get most of the conflict-miss benefit. **Every real CPU cache is set-associative**, typically 4-way, 8-way or 16-way.

Address split:

TAG	SET	WORD/OFFSET
-----	-----	-------------

Note the special cases: **1-way set associative = direct mapped**;
n-way where n = number of lines = fully associative.

Comparison table

	Direct	Fully associative	k-way set associative
A block can go in	1 specific line	Any line	Any of k lines in one set
Tag comparators needed	1	All lines (parallel)	k
Hardware cost	Lowest	Highest	Moderate
Search speed	Fastest	Slowest / most expensive	Fast
Conflict misses	Many (thrashing)	None	Few
Replacement algorithm needed?	No (only one choice)	Yes	Yes (within the set)
Tag field size	Smallest	Largest	Middle

Used in practice

Simple/embedded

TLBs, small buffers

All modern CPU caches

Worked example 4 — address partitioning for all three schemes

Given: Main memory = 64 KB (byte addressable). Cache = 2 KB. Block size = 16 bytes.

Find the address format for direct, associative and 4-way set-associative mapping.

STEP 1 — Basic quantities

Physical address size = $\log_2(64 \text{ K}) = \log_2(2^{16}) = 16 \text{ bits}$

Number of blocks in main memory = $64 \text{ KB} / 16 \text{ B} = 4096 = 2^{12}$

Number of lines in cache = $2 \text{ KB} / 16 \text{ B} = 128 = 2^7$

Word/offset bits = $\log_2(16) = 4 \text{ bits}$ ← same for all three schemes

(a) DIRECT MAPPING

Line (index) bits = $\log_2(128) = 7$

Tag bits = $16 - 7 - 4 = 5$

TAG 5	LINE 7	WORD 4
-------	--------	--------

= 16 bits ✓

Main memory block j maps to cache line $(j \bmod 128)$.

(b) FULLY ASSOCIATIVE MAPPING

Tag bits = $16 - 4 = 12$ (the tag is the full block number)

TAG 12	WORD 4
--------	--------

= 16 bits ✓

Needs 128 parallel 12-bit comparators.

(c) 4-WAY SET ASSOCIATIVE MAPPING

Number of sets = $128 \text{ lines} / 4 \text{ lines per set} = 32 = 2^5$

Set bits = 5

Tag bits = $16 - 5 - 4 = 7$

TAG 7	SET 5	WORD 4
-------	-------	--------

= 16 bits ✓

Main memory block j maps to set $(j \bmod 32)$, any of its 4 lines.

Summary table to reproduce:

Scheme	Tag	Line/Set	Word	Comparators
Direct	5	7 (line)	4	1
Fully associative	12	—	4	128
4-way set associative	7	5 (set)	4	4

Observation worth marks: as associativity increases, the tag grows and the index shrinks; the total is always 16.

Worked example 5 — which cache line does address 0x3A7C map to? (direct mapped)

Using the configuration above (16-bit address, tag 5 / line 7 / word 4):

0x3A7C = 0011 1010 0111 1100

Split: TAG (5) LINE (7) WORD (4)
 00111 0100111 1100
 = 7 = 39 = 12

→ Block number = 0x3A7C / 16 = 0x3A7 = 935. Check: 935 mod 128 = 935 - 7×128 = 935-896 = 39 ✓

→ It goes in cache LINE 39, with TAG 7, and the byte wanted is at OFFSET 12 within the block.

Replacement policies

Needed for associative and set-associative caches (direct mapping has no choice to make).

Policy	Rule	Pros	Cons
FIFO (First In First Out)	Evict the block that has been in the cache longest	Simple counter per line	Ignores usage; may evict a heavily-used block; suffers Belady's anomaly
LRU (Least Recently Used)	Evict the block unused for the longest time	Best practical performance — exploits temporal locality	Expensive to track exactly (needs age counters or a stack)
LFU (Least Frequently Used)	Evict the block with the smallest reference count	Good for stable working sets	An old, once-popular block can never be evicted; needs counters
Random	Evict a randomly chosen block	Zero bookkeeping cost	Unpredictable, but surprisingly close to LRU in practice
Optimal (MIN/OPT)	Evict the block that will be used furthest in the future	Theoretical minimum misses	Unimplementable — requires future knowledge. Used only as a benchmark

Pseudo-LRU (a tree of bits) is what real hardware actually uses for 8-way+ caches: approximate LRU at a fraction of the cost.

Worked example 6 — LRU vs FIFO on a 4-line fully associative cache

Block reference string: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

FIFO:

Ref	Cache contents (oldest → newest)	Hit/Miss
1	1	Miss
2	1 2	Miss
3	1 2 3	Miss
4	1 2 3 4	Miss

1	1 2 3 4	Hit
2	1 2 3 4	Hit
5	2 3 4 5 (evict 1)	Miss
1	3 4 5 1 (evict 2)	Miss
2	4 5 1 2 (evict 3)	Miss
3	5 1 2 3 (evict 4)	Miss
4	1 2 3 4 (evict 5)	Miss
5	2 3 4 5 (evict 1)	Miss

FIFO: 10 misses, 2 hits.

LRU:

Ref	Cache (LRU → MRU)	Hit/Miss
1	1	Miss
2	1 2	Miss
3	1 2 3	Miss
4	1 2 3 4	Miss
1	2 3 4 1	Hit
2	3 4 1 2	Hit
5	4 1 2 5 (evict 3, the LRU)	Miss
1	4 2 5 1	Hit
2	4 5 1 2	Hit
3	5 1 2 3 (evict 4)	Miss
4	1 2 3 4 (evict 5)	Miss
5	2 3 4 5 (evict 1)	Miss

LRU: 8 misses, 4 hits. LRU wins because it recognises that 1 and 2 are being reused.

Write policies

Policy	Mechanism	Advantage	Disadvantage
Write-through	Every write goes to both cache and main memory immediately	Main memory is always consistent ; simple; good for multiprocessors and DMA	High memory traffic — every write costs a slow memory access
Write-back (copy-back)	Write only to the cache; set a dirty bit ; write to memory only when the block is evicted	Much less memory traffic ; multiple writes to the same block cost one memory write	Memory is temporarily inconsistent (a <i>cache coherence</i> problem for DMA and other CPUs); more complex

for references that reach it) and main memory (access 100 ns). Accesses are hierarchical. Find the effective access time.

Work OUTWARD from the CPU:

Every access pays the L1 access time: 1 ns
 20% of accesses (miss L1) go on to L2: $0.20 \times [\text{L2 cost}]$
 $\text{L2 cost} = \text{L2 access} + (\text{L2 miss}) \times \text{memory}$
 $= 10 + 0.05 \times 100$
 $= 10 + 5 = 15 \text{ ns}$
 $T_{\text{eff}} = 1 + 0.20 \times 15 = 1 + 3 = 4 \text{ ns}$

Global miss rate = fraction of ALL references that reach main memory
 $= 0.20 \times 0.05 = 0.01 = 1\%$.

Local miss rate of L2 = $0.05 = 5\%$. (Distinguishing local from global miss rate is a favourite follow-up.)

Worked example 11 — the "how much faster" question.

A processor takes 1 cycle per instruction when all accesses hit the cache. The miss penalty is 50 cycles, the miss rate is 4%, and 30% of instructions also make a data reference. Find the effective CPI.

Memory accesses per instruction = 1 (instruction fetch) + 0.30 (data) = 1.30
 Stall cycles per instruction = $1.30 \times 0.04 \times 50 = 2.6$
 Effective CPI = $1 + 2.6 = 3.6$

The machine runs 3.6× slower than the ideal — a good illustration of why cache performance dominates real system performance.

Types of cache miss — the "three Cs" (a neat 5-mark answer)

Miss type	Cause	Cure
Compulsory (cold-start)	The very first reference to a block — it has never been in the cache	Larger blocks; prefetching
Capacity	The working set is larger than the whole cache	Larger cache
Conflict (collision)	Too many active blocks map to the same set, even though the cache isn't full	Higher associativity
(Coherence)	In multiprocessors, a block invalidated by another CPU's write	Coherence protocol tuning

Likely exam questions — Cache

Q	Marks
What is cache memory and why is it needed? Define hit ratio.	5
Explain the principle of locality of reference and its three forms.	5
Compare write-through and write-back cache policies.	5

Explain LRU, FIFO and Optimal replacement policies. Apply LRU and FIFO to the reference string 1,2,3,4,1,2,5,1,2,3,4,5 on a 4-line cache and compare the results.	10
A cache has 10 ns access time and main memory 100 ns. For hit ratios of 0.8, 0.9 and 0.95 compute the average access time and the speedup. What hit ratio gives an average access time of 15 ns?	10
Explain direct, fully associative and set-associative mapping with diagrams. For a 64 KB main memory, 2 KB cache and 16-byte blocks, derive the address format (tag/line/word) for each scheme and state the number of comparators required.	15
Explain the memory hierarchy. Discuss cache memory in detail — mapping techniques, replacement policies, write policies and performance evaluation — and work out the effective access time for a two-level cache with L1 (1 ns, 80% hit), L2 (10 ns, 95% hit) and main memory (100 ns).	15

MCQ traps — Cache

Trap	Truth
Cache is a type of secondary storage	It is fast SRAM , part of the primary memory hierarchy.
Cache is managed by the OS	Cache is managed entirely by hardware ; virtual memory is managed by the OS.
Direct mapping needs a replacement algorithm	It does not — there is only one possible line.
Fully associative mapping is used in modern CPU L2/L3	Too expensive; real caches are set-associative . Full associativity is used in TLBs .
Increasing block size always improves hit rate	Only up to a point — very large blocks reduce the number of blocks and increase the miss penalty (cache pollution).
Write-back keeps memory consistent	Write-through does. Write-back leaves memory stale until eviction.
Higher associativity always means faster	It reduces misses but increases the hit time (more comparators).
Dirty bit is used in write-through	It is used in write-back .
1-way set associative is fully associative	1-way = direct mapped .

D4. Memory interleaving

Concept

Even with a cache, when you *do* have to go to main memory you'd like to get several words at once — for example, to fill a whole cache block. **Memory interleaving splits main memory into several independent modules (banks) that can be accessed in an overlapping fashion.**

Because each module has a long recovery/cycle time but the bus is fast, you can start module 0, then immediately start module 1 while 0 is still working, and so on — pulling out consecutive words at bus speed instead of memory speed.

Two schemes

	Low-order (interleaved)	High-order (banked / consecutive)
Which address bits select the module	The least significant bits	The most significant bits
Consecutive addresses live in	Different modules	The same module
Benefit	Parallel access to consecutive addresses — ideal for sequential access, cache block fills and instruction streaming	Easy memory expansion; each module is a contiguous address range
Fault tolerance	A dead module punches holes throughout the address space	A dead module removes one contiguous region — the rest still works
Used for	Performance	Modularity

LOW-ORDER 4-WAY INTERLEAVING (address bits A1A0 select the module)

Module 0	Module 1	Module 2	Module 3
0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

Consecutive addresses spread ACROSS modules → all four can be busy at once.

HIGH-ORDER 4-WAY BANKING (top bits select the module)

Module 0	Module 1	Module 2	Module 3
0	4	8	12
1	5	9	13
2	6	10	14
3	7	11	15

Worked example 12 — interleaving speedup

A memory module has a cycle time of 100 ns and a bus transfer time of 10 ns. Compare the time to read 4 consecutive words from (a) a non-interleaved memory and (b) a 4-way low-order interleaved memory.

(a) NON-INTERLEAVED — each access must complete before the next begins:

$$4 \times 100 \text{ ns} = 400 \text{ ns}$$

(b) 4-WAY LOW-ORDER INTERLEAVED — all four modules start (staggered by the bus time) and run concurrently:

$$\begin{aligned}
 \text{Total} &= 100 \text{ ns (the first module's full cycle)} \\
 &+ 3 \times 10 \text{ ns (each remaining word arrives one bus transfer later)} \\
 &= 100 + 30 = 130 \text{ ns}
 \end{aligned}$$

$$\text{Speedup} = 400 / 130 \approx 3.08\times$$

With **n-way interleaving the ideal speedup approaches n**, provided the accesses are to consecutive addresses (i.e. no bank conflicts).

D5. Virtual memory

Concept

Two problems, one solution.

Problem 1: a program may be bigger than physical RAM. Historically programmers solved this by hand with "overlays" — manually swapping pieces in and out. Miserable.

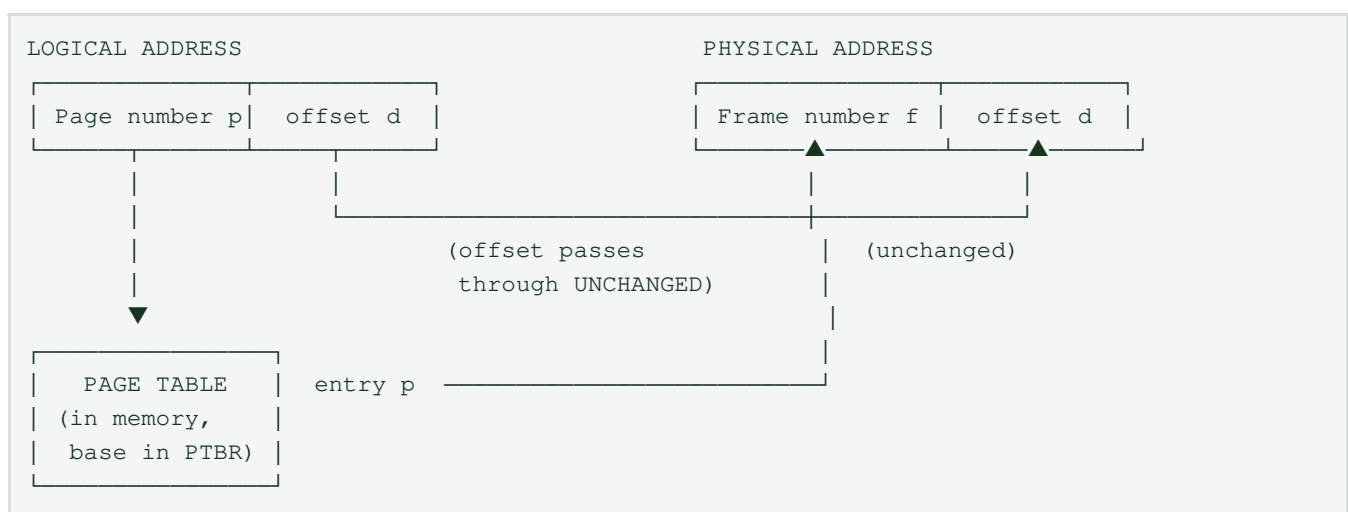
Problem 2: you want to run several programs at once, each believing it owns the whole address space, without them being able to corrupt each other.

Virtual memory solves both. Each process is given a large, private, contiguous **logical (virtual) address space**. The hardware (**MMU**) plus the OS translate every virtual address into a **physical address** at run time. Only the pages currently in use need be resident in RAM; the rest sit on disk and are fetched on demand. The programmer writes as if RAM were unlimited and private; the hardware maintains the illusion.

The essential point to state in an exam: virtual memory gives you (a) a logical address space larger than physical memory, (b) memory protection and isolation between processes, (c) relocation — a program can be loaded anywhere, and (d) efficient sharing of pages between processes. The cost is address-translation overhead and page-fault latency.

Paging

- The **logical address space** is divided into fixed-size **pages**.
- The **physical address space** is divided into equal-size **frames**.
- **Page size = frame size**, always, and always a **power of 2** (typically 4 KB).
- The **page table** maps page numbers to frame numbers, one per process.



Address arithmetic — the rules:

If page size = 2^n bytes, offset field = n bits
 If logical address = m bits, page number field = $(m - n)$ bits

```
Number of pages = 2m-n
Number of frames = physical memory size / page size
```

Worked example 13 — address field sizes.

A system has a 32-bit logical address, a 4 KB page size and 1 GB of physical memory.
Find the number of pages, the number of frames, the size of the offset, and the page table size if each entry is 4 bytes.

```
Page size = 4 KB = 212 → offset = 12 bits
Page number field = 32 - 12 = 20 bits
Number of pages = 220 = 1,048,576 pages

Physical memory = 1 GB = 230 → physical address = 30 bits
Frame number field = 30 - 12 = 18 bits
Number of frames = 218 = 262,144 frames

Page table size = 220 entries × 4 bytes = 4 MB PER PROCESS
```

That 4 MB figure is the point of the question — the page table is far too big to keep in registers and too big to want a copy per process in RAM. It motivates **multilevel page tables**, **inverted page tables** and, above all, the **TLB**.

Worked example 14 — actual address translation.

Page size 4 KB. The page table shows page 3 → frame 10 (0x00A). Translate logical address 0x00003F7A.

```
Logical address 0x00003F7A = 0000 0000 0000 0000 0011 1111 0111 1010

Offset = low 12 bits = 1111 0111 1010 = 0xF7A
Page number = high 20 bits = 0x00003 = 3

Page 3 → Frame 0x00A (from the page table)

Physical address = frame number || offset
                  = 0x00A || 0xF7A
                  = 0x00AF7A

Sanity check in decimal: frame 10 × 4096 = 40960; offset 0xF7A = 3962
                        40960 + 3962 = 44922 = 0xAF7A ✓
```

Page table entry contents

Field	Purpose
Frame number	Where the page lives in physical memory
Valid / Present bit	Is the page currently in RAM? If 0 → page fault
Dirty / Modified bit	Has it been written? If not, no need to write it back on eviction
Reference / Accessed bit	Used by LRU-approximation replacement algorithms
Protection bits	Read / Write / Execute permissions

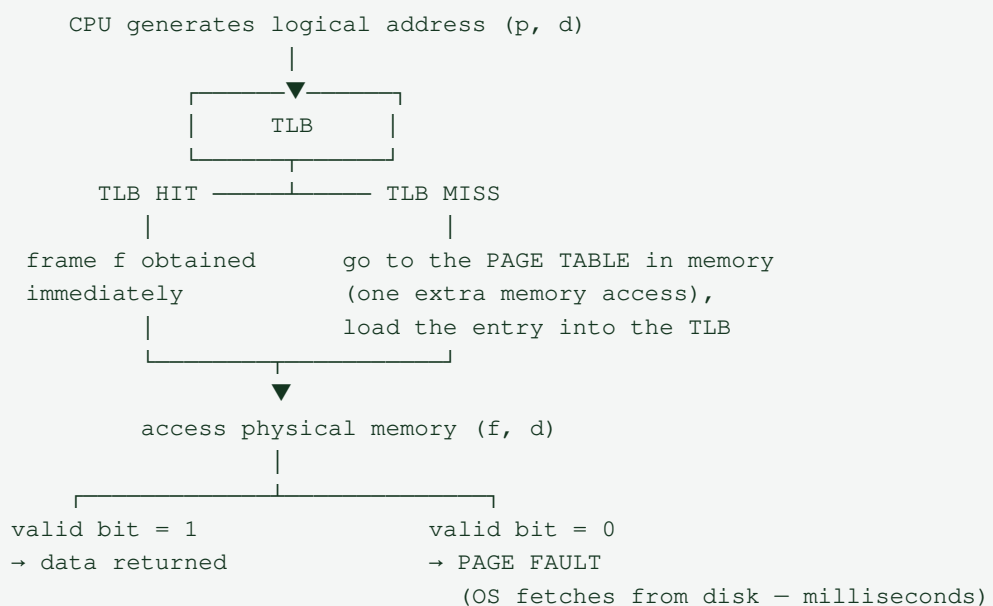
Caching disabled bit

For memory-mapped I/O regions

The TLB (Translation Lookaside Buffer)

The problem it solves: without help, every single memory reference now takes **two** memory accesses — one to read the page table entry and one to get the actual data. That halves your memory performance.

The solution: the TLB is a **small, fully associative, very fast hardware cache of recently-used page-table entries** (typically 64–1024 entries), sitting inside the MMU. Thanks to locality, the hit rate is usually 98–99%.



Effective Access Time with a TLB:

$$\begin{aligned} \text{EAT} &= h(t_{\text{TLB}} + t_{\text{mem}}) + (1 - h)(t_{\text{TLB}} + 2 \cdot t_{\text{mem}}) \\ &= t_{\text{TLB}} + t_{\text{mem}} + (1 - h) \cdot t_{\text{mem}} \end{aligned}$$

(TLB hit: look in TLB, then one memory access for data.

TLB miss: look in TLB, one access for the page table, one for the data.)

Worked example 15 — TLB effective access time.

TLB access = 20 ns, main memory access = 100 ns, TLB hit ratio = 80%. Find the EAT.

$$\begin{aligned} \text{TLB hit (80\%): } & 20 + 100 = 120 \text{ ns} \\ \text{TLB miss (20\%): } & 20 + 100 + 100 = 220 \text{ ns} \end{aligned}$$

$$\begin{aligned} \text{EAT} &= 0.80 \times 120 + 0.20 \times 220 \\ &= 96 + 44 \\ &= 140 \text{ ns} \end{aligned}$$

Compare: with no paging at all it would be 100 ns. Slowdown = 40%.

If the hit ratio rises to 98%:

$$\text{EAT} = 0.98 \times 120 + 0.02 \times 220 = 117.6 + 4.4 = 122 \text{ ns} \rightarrow \text{slowdown only 22\%}.$$

(Some textbooks omit the TLB access time on a hit, giving $\text{EAT} = 0.8 \times 100 + 0.2 \times 200 = 120 \text{ ns}$. Always state which convention you are using, then you cannot be marked wrong.)

Page faults

A **page fault** occurs when the valid bit is 0 — the page is not in physical memory. The sequence:

1. MMU raises a page-fault trap → control transfers to the OS
2. OS checks the reference is legal (else → segmentation fault, kill the process)
3. Find a free frame; if none, run the PAGE REPLACEMENT algorithm to evict one (write it back to disk first if the dirty bit is set)
4. Schedule a disk read of the required page into the frame
5. The process **BLOCKS**; the CPU runs something else meanwhile
6. On disk-interrupt: update the page table (valid bit = 1, frame number), update the TLB
7. RESTART the instruction that faulted

Effective access time with page faults:

$$\text{EAT} = (1 - p) \times \text{memory_access} + p \times \text{page_fault_service_time}$$

Worked example 16. Memory access 100 ns, page fault service 10 ms = 10,000,000 ns.
Page fault rate $p = 0.001$.

$$\begin{aligned}\text{EAT} &= 0.999 \times 100 + 0.001 \times 10,000,000 \\ &= 99.9 + 10,000 \\ &= 10,099.9 \text{ ns} \approx 10.1 \mu\text{s}\end{aligned}$$

That is 101× SLOWER than raw memory, from a fault rate of only 1 in 1000.
For less than 10% degradation you need $p < 1$ in ~1,000,000.

This calculation is the single best justification for why page-replacement algorithms matter — quote it. (Page replacement algorithms themselves are worked in detail in **CORE_03_OperatingSystems.md §F**.)

Segmentation and segmented paging

	Paging	Segmentation
Division unit	Fixed-size pages	Variable-size segments
Chosen by	The hardware/OS — invisible to the programmer	The programmer/compiler — logical units (code, data, stack, heap)
Address form	Page number + offset (one-dimensional logical address)	Segment number + offset (two-dimensional)
Fragmentation	Internal (last page partly unused)	External (gaps between segments)
Table entry	Frame number	Base + limit
Protection/sharing	Per page — less natural	Per segment — natural (a whole procedure or data structure)
Compaction needed?	No	Sometimes

Segmented paging combines both: divide the program into segments, then divide each segment into pages. You get the logical structure of segmentation with the fragmentation

behaviour of paging. Used by x86.

Segmentation address translation: physical = base[s] + d, **but first check d < limit[s]**, else raise a trap. The limit check is what provides protection.

Likely exam questions — Virtual memory

Q	Marks
What is virtual memory? State four benefits.	5
What is a TLB and why is it needed?	5
Distinguish between paging and segmentation.	5
Explain address translation in a paged system with a diagram. A system has a 32-bit logical address and 4 KB pages; find the number of pages, the offset size and the page table size if each entry is 4 bytes.	10
A TLB has an access time of 20 ns, memory 100 ns and a hit ratio of 80%. Compute the effective access time. Recompute for 98% and comment.	10
Explain virtual memory in detail: the motivation, paging, the page table and its entry fields, the TLB, the page fault handling sequence, and effective access time. Show with a numerical example why a low page-fault rate is essential.	15

MCQ traps — Virtual memory

Trap	Truth
Virtual memory increases the speed of a computer	It increases the apparent size of memory. It generally decreases speed slightly (translation + faults).
Page size can be any value	Must be a power of 2 so the address splits cleanly with no division.
Page and frame sizes may differ	They are always equal .
The offset is translated too	The offset passes through unchanged . Only the page number is translated.
TLB is a cache of data	It is a cache of page table entries (address translations).
TLB is direct mapped	It is typically fully associative (small enough to afford it).
Paging causes external fragmentation	Paging causes internal fragmentation; segmentation causes external.
Page fault = hardware error	It is a normal, expected event handled by the OS.
A page fault is handled by hardware	The trap is raised by hardware; the handling is done by the OS .

PART E — I/O ORGANISATION

E1. I/O addressing: memory-mapped vs isolated

Concept

An I/O device controller has registers — a data register, a status register, a control register. The CPU has to read and write them. **The question is: do those registers live in the same address space as memory, or in a separate one?**

	Memory-mapped I/O	Isolated (I/O-mapped / port-mapped) I/O
Address space	I/O registers occupy part of the normal memory address space	I/O has its own separate address space
Instructions used	Ordinary memory instructions (LOAD, STORE, MOV)	Special I/O instructions (IN, OUT)
Control signals	Only MEMR / MEMW	Needs extra IOR / IOW signals (an M/I $\bar{O}$ line)
Address lines used	All of them	Usually fewer (e.g. 8 lines → 256 ports)
Memory available to programs	Reduced — I/O steals address space	Full memory space available
Programming flexibility	High — the full instruction set (arithmetic, logical, bit operations) works on device registers	Limited — only IN/OUT, usually via the accumulator
Decoding hardware	More address lines to decode	Simpler
Used by	Motorola 68000, ARM, MIPS, PowerPC, most RISC	Intel x86 (which supports both)

Linear selection (named in the CF syllabus): a simplified decoding technique where each device's chip-select is driven directly by **one dedicated address line** rather than by a decoder. Cheap, but only n devices for n address lines, and large parts of the address space become unusable/aliased.

E2. The three data transfer techniques

Concept

A disk is a million times slower than a CPU. How do you move data between them without wasting all your CPU cycles? Three answers, in increasing order of sophistication:

1. **Programmed I/O**: the CPU personally asks the device "are you ready? are you ready?" over and over, then moves each word itself. Simple, and a total waste of the CPU.
2. **Interrupt-driven I/O**: the CPU says "tell me when you're ready" and goes off to do other work; the device raises an interrupt. Much better CPU utilisation, but the CPU still personally moves every single word.
3. **DMA**: the CPU says "here's the buffer address and the count — move the whole block yourself and interrupt me when the *entire transfer* is done." The DMA controller becomes a bus master and moves the data directly between the device and memory. **The CPU is involved only twice — at setup and at completion.** This is how all high-speed I/O works.

Programmed I/O (polling)

```
LOOP:  read the device STATUS register
       is the READY/BUSY flag set?
       NO  → jump back to LOOP          ← BUSY-WAITING / POLLING
       YES → read the DATA register, store it to memory
           decrement the count; if not zero, jump to LOOP
```

- The CPU is **100% occupied** and does no useful work while waiting.
- Simple, no extra hardware, deterministic timing.
- Acceptable only for very slow, dedicated, or embedded systems.

Interrupt-driven I/O

```
1. CPU issues the I/O command and continues with other work
2. When the device is ready, it asserts the INTERRUPT REQUEST line
3. CPU finishes the current instruction, then:
   pushes PC and PSW onto the stack
   identifies the interrupting device
   loads the ISR address (from the interrupt vector table)
4. The ISR transfers ONE word (or a small burst) between the device and memory
5. IRET restores PC and PSW → the interrupted program resumes
```

- CPU utilisation is much better, but there is **overhead per word** (context save/restore is dozens of cycles), so for a 4 KB disk block this is still very expensive.

Identifying the interrupting device — four methods:

Method	How
Software polling	The ISR reads each device's status register in turn. Simple but slow; poll order = priority
Daisy chaining	Devices are wired in series on the interrupt-acknowledge line; the acknowledge signal propagates down the chain and the first device that wants service grabs it . Priority = physical position in the chain . Simple hardware
Vectored interrupts	The device places its own identifying vector (or the ISR address) on the data bus during the acknowledge cycle → the CPU jumps straight to the right ISR. Fast
Priority interrupt controller (e.g. 8259 PIC, APIC)	A dedicated chip arbitrates among multiple requests and supplies the vector

Interrupt classification (worth a 5-mark bullet list):

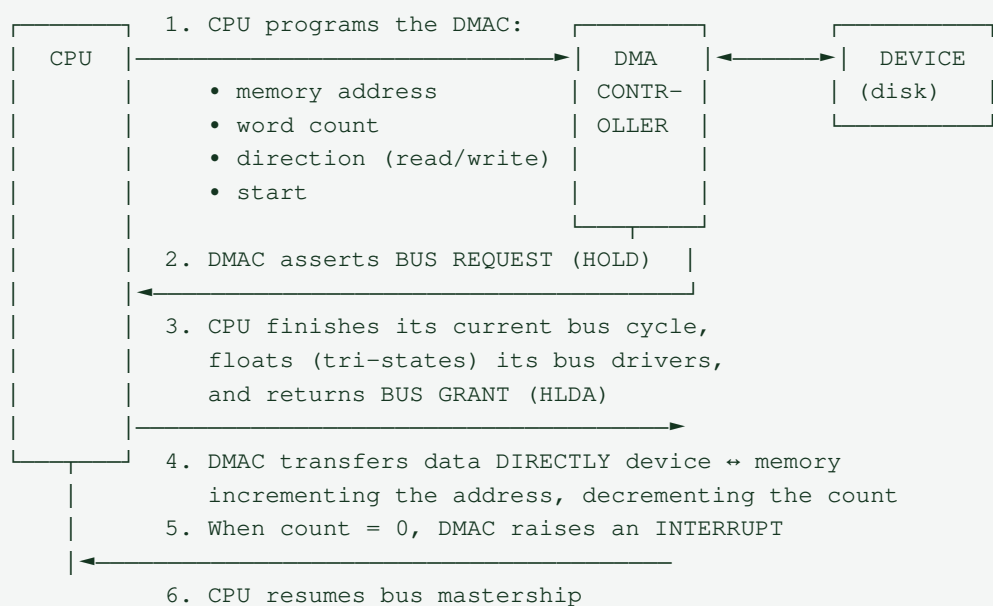
Class	Examples
Hardware vs Software	Device signal vs. an INT/TRAP instruction (system call)
Maskable vs Non-maskable (NMI)	Can be disabled vs. cannot (power failure, memory parity error)
Vectored vs Non-vectored	Device supplies the ISR address vs. a fixed ISR address

Internal (traps/exceptions) vs External	Divide-by-zero, page fault, invalid opcode vs. keyboard, timer, disk
Synchronous vs Asynchronous	Caused by the executing instruction vs. arriving at any time

DMA (Direct Memory Access)

The core idea: the DMA controller (DMAC) temporarily takes over the system bus from the CPU and moves data **directly between the I/O device and memory, bypassing the CPU entirely**.

DMA SETUP AND TRANSFER (hand-draw this sequence)



DMA controller registers: Memory Address Register, Word Count Register, Control Register (direction, mode), Address/Data bus buffers.

The three DMA transfer modes:

Mode	Behaviour	CPU impact	Use
Burst mode (block transfer)	The DMAC seizes the bus and transfers the entire block without releasing it	CPU is completely blocked for the duration	Fastest transfer ; used for disk blocks
Cycle stealing	The DMAC transfers one word at a time , releasing the bus in between; it "steals" occasional bus cycles from the CPU	CPU is only slightly slowed	The most common mode — good balance
Transparent (hidden) DMA	The DMAC transfers only during cycles when the CPU is not using the bus (e.g. internal ALU cycles)	No CPU slowdown at all	Slowest transfer; needs complex detection logic

Cycle stealing does NOT interrupt the CPU — it just delays it by one bus cycle. No context is saved or restored, which is exactly why it is so much cheaper than interrupt-driven transfer. **This distinction is a classic MCQ.**

The three-way comparison — the table to reproduce

	Programmed I/O	Interrupt-driven I/O	DMA
Data transferred by	CPU	CPU (inside the ISR)	DMA controller
Data path	Device → CPU → Memory	Device → CPU → Memory	Device ↔ Memory directly
CPU waits/polls?	Yes — busy-waits	No	No
CPU involvement	Continuous	Per word/byte	Only at start and end of the whole block
Interrupts used	No	Yes, per word	Yes, once per block
Transfer speed	Slowest	Medium	Fastest
Extra hardware	None	Interrupt logic	DMA controller (expensive)
CPU utilisation	Very poor	Good	Excellent
Suitable for	Slow devices, simple embedded systems	Moderate-speed devices (keyboard, mouse, serial)	High-speed block devices (disk, network, graphics, sound)

"DMA is used to reduce CPU intervention in data transfer" — that one sentence answers a large fraction of the MCQs on this topic.

E3. Buses

Synchronous vs asynchronous buses

	Synchronous bus	Asynchronous bus
Timing	Governed by a common clock ; every event happens at a fixed clock edge	No clock — events are sequenced by handshaking signals
Speed of devices	All transfers take the same time — must be paced to the slowest device	Each transfer takes exactly as long as that device needs
Signals	Clock, address, data, read/write	Master-ready / Slave-ready (or REQ/ACK) handshake lines
Complexity	Simple control logic	More complex (handshake circuitry)
Bus length	Limited — clock skew over long distances	Can be longer
Mixing fast and slow devices	Inefficient	Efficient
Examples	Processor–memory buses, PCI	SCSI, older I/O buses

Asynchronous handshake — the four-phase (fully interlocked) protocol:

1. Master places the address and asserts MASTER-READY
2. Slave decodes, prepares the data, and asserts SLAVE-READY
3. Master reads the data and de-asserts MASTER-READY
4. Slave de-asserts SLAVE-READY

Each edge is a response to the previous one – hence "fully interlocked".
The transfer takes exactly as long as it needs. No clock is involved.

Bus arbitration (deciding who becomes bus master when several want it):

Method	How
Daisy chaining	Grant signal propagates serially; nearest device wins. Simple, but fixed priority and a fault breaks the chain
Polling	The arbiter polls each device in turn using a count on the poll lines. Priority is programmable
Independent request	Each device has its own request and grant line into a central arbiter. Fastest , but the most wires
Centralised vs distributed	One arbiter chip vs. devices resolving among themselves

Standard interface buses (both CF and Diploma syllabi mention these)

Bus	Type	Notes
ISA	Parallel, 8/16-bit, 8 MHz	Legacy PC expansion bus
PCI	Parallel, 32/64-bit, 33/66 MHz, ~133–533 MB/s	Plug and play : processor-independent; was the standard expansion bus
AGP	Accelerated Graphics Port	Dedicated point-to-point port for the graphics card, direct access to system RAM. Superseded by PCIe
PCI Express (PCIe)	Serial , point-to-point lanes (×1, ×4, ×8, ×16), full duplex	The current standard for graphics, SSDs (NVMe), and expansion
USB	Serial, tiered-star topology, hot-pluggable , up to 127 devices	USB 1.1 = 12 Mbps, 2.0 = 480 Mbps, 3.0 = 5 Gbps, 3.1 = 10 Gbps, 4 = 40 Gbps. Also supplies power
IDE / PATA	Parallel ATA, 40/80-pin ribbon, 2 drives per channel (master/slave)	Legacy disk interface
SATA	Serial ATA, point-to-point, thin cable, hot-swap	SATA I 1.5 Gbps, II 3 Gbps, III 6 Gbps. Current disk standard
SCSI / SAS	Parallel / Serial Attached SCSI	Servers; supports many devices per bus, high performance
Serial port (RS-232)	One bit at a time	9-pin D-sub; legacy

Parallel port (Centronics)	8 bits at a time	25-pin; legacy printers
PS/2	6-pin mini-DIN	Legacy keyboard (purple) and mouse (green)
HDMI / DisplayPort	Digital audio-video	Modern display interfaces

Serial vs parallel — why serial won. At low speeds parallel is faster (8 bits at once). But at high speeds, **skew** (the bits of a parallel word arriving at slightly different times) and **crosstalk** between adjacent wires become limiting. A single, well-shielded, differential serial line can be clocked *far* higher, and you can add lanes for bandwidth. Hence PATA→SATA, PCI→PCIe, parallel port→USB. **This is a good 5-mark answer.**

Likely exam questions — I/O organisation

Q	Marks
Distinguish between memory-mapped I/O and isolated I/O.	5
What is DMA? Why is it used?	5
Explain the three DMA transfer modes.	5
List the methods of identifying an interrupting device and explain daisy chaining.	5
Compare programmed I/O, interrupt-driven I/O and DMA under at least seven headings.	10
Explain DMA data transfer with a block diagram. Describe the DMA controller registers and the complete transfer sequence including bus request and bus grant.	10
Compare synchronous and asynchronous buses. Explain the four-phase handshaking protocol with a timing description.	10
Discuss I/O organisation in a computer system: I/O addressing techniques (memory-mapped, isolated, linear selection), the three data transfer methods with their relative merits, interrupt handling and priority resolution, and DMA including its modes.	15

MCQ traps — I/O organisation

Trap	Truth
In DMA, data passes through the CPU	No — that is the entire point. Data goes directly between the device and memory.
DMA needs no CPU involvement at all	The CPU initialises the DMAC and handles the completion interrupt .
Cycle stealing interrupts the CPU	It does not interrupt — it steals a bus cycle, delaying the CPU by one cycle.
Burst mode is slowest	Burst is the fastest transfer ; transparent DMA is the slowest but least intrusive.
Memory-mapped I/O uses IN/OUT instructions	Memory-mapped uses ordinary LOAD/STORE . IN/OUT belong to isolated I/O .

INSTRUCTION COMPLETES EVERY CYCLE.

F2. Performance formulas — memorise these

Non-pipelined time for n instructions: $T_{np} = n \times k \times t$

Pipelined time for n instructions: $T_p = (k + n - 1) \times t$

where k = number of stages, t = clock period (one stage time)

SPEEDUP $S = T_{np} / T_p = (n \times k) / (k + n - 1)$

MAXIMUM SPEEDUP (as $n \rightarrow \infty$) $S_{max} = k$ ← equals the number of stages

EFFICIENCY $\eta = S / k = n / (k + n - 1)$

THROUGHPUT $= n / [(k + n - 1) \times t]$ instructions per unit time
→ $1/t$ in the steady state

CLOCK PERIOD of a pipeline $t = (\text{slowest stage delay}) + (\text{latch delay})$

Worked example 17 — basic pipeline speedup

A 5-stage pipeline has a clock period of 2 ns. Compute the time and speedup for 100 instructions.

Non-pipelined: $T = 100 \times 5 \times 2 = 1000$ ns

Pipelined: $T = (5 + 100 - 1) \times 2 = 104 \times 2 = 208$ ns

Speedup $= 1000 / 208 = 4.81$

Efficiency $= 4.81 / 5 = 0.962 = 96.2\%$

Throughput $= 100 / 208 \text{ ns} = 0.481 \text{ instructions/ns} = 481 \text{ MIPS}$

Note: the theoretical maximum speedup is 5; we achieve 4.81 because of pipeline fill-up and drain overhead. As n grows, the speedup approaches 5.

Worked example 18 — unequal stage delays (the harder version)

An instruction pipeline has stage delays of 5, 7, 4, 6 and 3 ns. Each inter-stage latch adds 1 ns. Find (a) the clock period, (b) the non-pipelined execution time per instruction, (c) the speedup for a large number of instructions, (d) the time for 1000 instructions.

(a) Clock period = slowest stage + latch = $7 + 1 = 8$ ns
(Every stage must be given the same time, set by the SLOWEST — this is why balancing pipeline stages matters.)

(b) Non-pipelined time per instruction = $5 + 7 + 4 + 6 + 3 = 25$ ns (no latches needed)

(c) Speedup ($n \rightarrow \infty$) = $25 / 8 = 3.125$

Note this is LESS than the ideal 5, purely because the stages are unbalanced and the latches add overhead. Balancing the stages to 5 ns each would give a clock of 6 ns and a speedup of $25/6 = 4.17$.

(d) $T = (5 + 1000 - 1) \times 8 = 1004 \times 8 = 8032$ ns

Non-pipelined: $1000 \times 25 = 25,000$ ns. Actual speedup = $25000/8032 = 3.11$

F3. Pipeline hazards — the three classes

A **hazard** is any condition that prevents the next instruction from executing in its designated clock cycle.

1. Structural hazards (resource conflicts)

Cause: two instructions need the **same hardware resource** in the same cycle. Classic case: a single unified memory — instruction I1 in MEM wants memory while I4 in IF also wants memory.

Solutions:

- **Duplicate the resource** — separate instruction and data caches (a Harvard-style split L1). This is the standard fix.
- **Stall** (insert a pipeline bubble) — always works, always costs cycles.
- **Pipeline the resource** — e.g. a pipelined multiplier.

2. Data hazards (dependency conflicts)

Cause: an instruction needs a result that a previous instruction has not yet produced.

```
I1:  ADD  R1, R2, R3      ← R1 is written in WB (cycle 5)
I2:  SUB  R4, R1, R5      ← R1 is READ in ID (cycle 3) → R1 IS NOT READY YET

Cycle:    1    2    3    4    5
I1:       IF   ID   EX   MEM   WB      ← R1 written here
I2:              IF   ID*   EX   MEM    ← R1 needed here (cycle 3) ✗
                ↑ HAZARD
```

The three dependency types:

Type	Name	Pattern	Occurs in a simple in-order pipeline?
RAW	Read After Write — <i>true dependency</i>	I2 reads what I1 writes	Yes — this is the common one
WAR	Write After Read — <i>anti-dependency</i>	I2 writes what I1 reads	Only with out-of-order execution
WAW	Write After Write — <i>output dependency</i>	Both write the same register	Only with out-of-order / multi-cycle stages

Solutions to data hazards:

Solution	How it works
Operand forwarding / bypassing	The main hardware fix. Route the ALU result directly from the EX/MEM latch back to the ALU input of the next instruction, <i>before</i> it is written to the register file. Eliminates most RAW stalls with zero cycle penalty
Pipeline interlock / stalling	Hardware detects the dependency and inserts bubbles until the data is ready. Always works, costs cycles

Compiler instruction reordering	The compiler moves independent instructions into the gap
NOP insertion	The compiler inserts no-operations — simple but wastes cycles
Register renaming	Eliminates WAR and WAW by using different physical registers

Forwarding cannot eliminate the LOAD-USE hazard. If I1 is `LW R1, 0(R2)` the data only arrives at the end of MEM (cycle 4), but a dependent I2 needs it at the start of its EX (cycle 4). **One stall cycle is unavoidable**, then forwarding covers the rest. This is a favourite advanced MCQ.

3. Control hazards (branch hazards)

Cause: the pipeline fetches instructions sequentially, but a **branch** may redirect execution. Until the branch outcome is known (typically at the end of EX), the instructions already fetched behind it may be wrong and must be **flushed**.

Branch penalty = the number of cycles lost = the number of instructions that must be flushed.

Solutions:

Solution	How it works
Stall until resolved	Simple, maximum penalty
Branch prediction — static	Always predict not-taken (just keep fetching); or "backward taken, forward not taken" (loops branch backward and are usually taken)
Branch prediction — dynamic	A Branch History Table / Branch Target Buffer records what each branch did last time. 2-bit saturating counters are standard (a branch must mispredict twice to flip the prediction, so a loop's single exit doesn't poison it). Modern predictors exceed 95% accuracy
Delayed branch	The ISA defines the instruction(s) <i>after</i> the branch as always executed (the branch delay slot); the compiler fills the slot with useful work. Used in MIPS and SPARC
Branch target buffer	Caches the target address so the pipeline can start fetching immediately
Early branch resolution	Move the comparison hardware into the ID stage, reducing the penalty from 3 cycles to 1
Speculative execution	Execute down the predicted path and discard the results if wrong

Worked example 19 — effective CPI with hazards

A 5-stage pipeline has an ideal CPI of 1. 20% of instructions are branches with a 3-cycle misprediction penalty and a 10% misprediction rate. 30% of instructions cause a load-use stall of 1 cycle. Find the effective CPI and the resulting speedup over a non-pipelined

machine that takes 5 cycles per instruction.

```
Branch stall cycles per instruction = 0.20 × 0.10 × 3 = 0.06
Load-use stall cycles per instruction = 0.30 × 1           = 0.30
Total stalls per instruction          = 0.36
```

Effective CPI = 1 + 0.36 = 1.36

Speedup over non-pipelined = 5 / 1.36 = 3.68
(instead of the ideal 5 – hazards cost us 26% of the theoretical benefit)

F4. Types of pipeline and related terms

Term	Meaning
Instruction pipeline	Overlaps the phases of instruction execution (the 5-stage pipeline above)
Arithmetic pipeline	Splits a single arithmetic operation into stages (floating-point add: compare exponents → align mantissas → add → normalise). Used in vector processors
Superscalar	Multiple pipelines in parallel — more than one instruction issued per cycle. CPI can be less than 1
Superpipelined	A deeper pipeline (more, shorter stages) allowing a higher clock rate
VLIW	Very Long Instruction Word — the compiler packs independent operations into one wide instruction; the hardware issues them in parallel with no dependency-checking logic
Pipeline bubble / stall	A cycle in which a stage does nothing, propagating down the pipeline
Flushing	Discarding wrongly-fetched instructions after a mispredicted branch

Amdahl's Law (worth one line): the overall speedup from improving one part of a system is limited by the fraction of time that part is used.

$$\text{Speedup}_{\text{overall}} = 1 / [(1 - f) + f/s]$$

where f is the fraction improved and s is the speedup of that fraction. If 60% of a program is parallelisable and you make that part infinitely fast, the maximum overall speedup is $1/0.4 = 2.5\times$. This is the fundamental limit on all parallel architecture.

Likely exam questions — Pipelining

Q	Marks
What is pipelining? State the five stages of a typical instruction pipeline.	5
Define speedup, efficiency and throughput of a pipeline and give the formulas.	5

What is a data hazard? Explain RAW, WAR and WAW dependencies.	5
A 5-stage pipeline has stage delays 5, 7, 4, 6 and 3 ns with 1 ns latches. Find the clock period, the speedup for a large n, and the time for 1000 instructions.	10
Explain the three classes of pipeline hazard and the techniques used to overcome each.	10
Explain pipelining with a space-time diagram. Derive the speedup formula and show that the maximum speedup equals the number of stages. Discuss structural, data and control hazards with examples and their remedies, and explain why operand forwarding cannot fully eliminate a load-use hazard.	15

MCQ traps — Pipelining

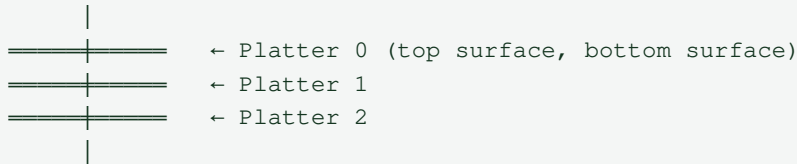
Trap	Truth
Pipelining reduces the execution time of a single instruction	It does not — it increases throughput . Latency per instruction stays the same or worsens slightly.
A k-stage pipeline gives exactly $k\times$ speedup	Only asymptotically and only with no hazards; the fill/drain overhead and stalls reduce it.
Pipeline clock period = average stage delay	It equals the slowest stage delay + latch delay.
RAW hazards can be fully removed by forwarding	Not the load-use case — one stall is unavoidable.
WAR and WAW occur in a simple in-order pipeline	They appear with out-of-order execution.
Superscalar and superpipelined are the same	Superscalar = multiple pipelines (issue width > 1). Superpipelined = more stages (deeper).
Branch prediction is done by the compiler only	Dynamic prediction is done by hardware at run time.
More stages is always better	Deeper pipelines have larger branch penalties and more latch overhead — there is an optimum.

PART G — STORAGE AND HARDWARE (Diploma-focused)

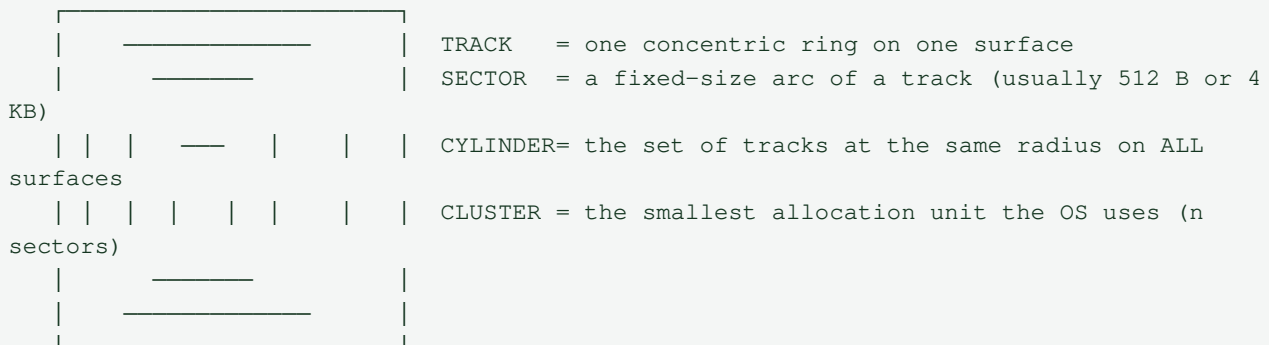
G1. Magnetic disk (hard disk)

Geometry — the vocabulary

SPINDLE



Read/write HEADS on an ACTUATOR ARM, one per surface,
 all moving TOGETHER



Capacity formula:

Capacity = (number of surfaces) × (tracks per surface) × (sectors per track) × (bytes per sector)
 number of surfaces = 2 × number of platters

Worked example 20 — disk capacity.

A disk has 4 platters, 2000 tracks per surface, 63 sectors per track and 512 bytes per sector. Find its capacity.

Surfaces = $4 \times 2 = 8$
 Capacity = $8 \times 2000 \times 63 \times 512$ bytes
 $= 8 \times 2000 = 16,000$ tracks
 $16,000 \times 63 = 1,008,000$ sectors
 $1,008,000 \times 512 = 516,096,000$ bytes
 ≈ 492 MiB (or 516 MB using the decimal convention)

Access time — the three components

DISK ACCESS TIME = SEEK TIME + ROTATIONAL LATENCY + TRANSFER TIME (+ controller overhead)

Component	Meaning	Typical
Seek time	Time to move the head to the correct cylinder	3–10 ms — the dominant and most variable component
Rotational latency	Time to wait for the required sector to rotate under the head	Average = half a revolution

Transfer time	Time to read/write the data as it passes under the head	(bytes / bytes-per-track) × revolution time
----------------------	---	---

Average rotational latency = (1/2) × (60 / RPM) seconds

RPM	Time per revolution	Average latency
5400	11.1 ms	5.56 ms
7200	8.33 ms	4.17 ms
10,000	6 ms	3 ms
15,000	4 ms	2 ms

Worked example 21 — disk access time.

A disk rotates at 7200 RPM, has an average seek time of 5 ms, 500 sectors per track and 512 bytes per sector. Find the average time to read one sector.

```

Time per revolution = 60 / 7200 s = 8.33 ms
Average rotational latency = 8.33 / 2 = 4.17 ms
Transfer time for 1 sector = 8.33 ms / 500 sectors = 0.0167 ms

Total = 5 + 4.17 + 0.017 = 9.19 ms

Data transfer rate = 500 × 512 bytes per 8.33 ms = 256,000 B / 0.00833 s
                  ≈ 30.7 MB/s

```

The point to state: seek + latency = 9.17 ms, transfer = 0.017 ms. **Over 99% of the time is spent positioning, not transferring.** That is why disk scheduling algorithms (covered in CORE_03 §G) optimise *seek* time, and why sequential access is enormously faster than random access.

G2. Optical and solid-state storage

Medium	Capacity	Mechanism
CD-ROM	700 MB	Infrared 780 nm laser reads pits and lands on a spiral track from the centre outward
CD-R	700 MB	Write-once; the laser burns a dye layer
CD-RW	700 MB	Rewritable; phase-change material toggled between crystalline and amorphous states
DVD	4.7 GB (single layer) / 8.5 GB (dual layer) / 17 GB (dual-sided dual-layer)	650 nm red laser, tighter track pitch
Blu-ray	25 GB / 50 GB dual layer	405 nm blue-violet laser — shorter wavelength → smaller pits → more capacity

Why blue gives more capacity: the minimum readable pit size is limited by the laser wavelength (diffraction). Shorter wavelength → smaller spot → smaller pits → more data.
Neat one-sentence answer.

SSD (Solid State Drive): NAND flash, **no moving parts**.

	HDD	SSD
Moving parts	Yes (platters, heads)	None
Access time	~5–10 ms	~0.05–0.1 ms
Random access performance	Poor (seek dominated)	Excellent — no seek time at all
Shock resistance / noise / power	Poor / noisy / higher	Excellent / silent / lower
Cost per GB	Low	Higher
Endurance	Effectively unlimited writes	Limited write cycles per cell → needs wear levelling
Data recovery after deletion (forensic note)	Data usually persists until overwritten	TRIM can zero blocks quickly → much harder forensic recovery

That last row is a genuinely examinable point in the **Computer Forensic** paper: TRIM and wear levelling make SSD forensics fundamentally different from HDD forensics.

G3. Motherboard components (CS Diploma Paper-I §4, verbatim topics)

Component	Function
Chipset — Northbridge / Memory Controller Hub (MCH)	Historically connected the CPU to the fastest subsystems: system bus (FSB), RAM (DIMM slots), and the AGP/PCIe graphics port . In modern CPUs the memory controller has been moved onto the CPU die , so the northbridge has largely disappeared.
Chipset — Southbridge / I/O Controller Hub (ICH)	Connects the slower peripherals: IDE/SATA, PCI, USB, serial and parallel ports, PS/2, audio, LAN, BIOS ROM
Processor socket	Physical/electrical CPU mount (LGA — pins on the socket; PGA — pins on the chip)
DIMM slots	Memory module sockets; usually colour-coded in pairs for dual-channel operation
AGP slot	Legacy dedicated graphics port (superseded by PCIe ×16)
PCI / PCIe slots	Expansion cards
IDE (Primary/Secondary) and SATA connectors	Disk and optical drive connections

BIOS ROM	Non-volatile chip holding the BIOS / UEFI firmware : POST, bootstrap loader, low-level hardware routines
CMOS chip + CMOS battery (CR2032)	Small volatile RAM holding BIOS settings, date and time ; the lithium battery keeps it powered when the PC is off . Removing the battery resets BIOS settings and clears the BIOS password — a well-known forensic/technician fact
RTC (Real Time Clock)	Keeps time continuously; battery-backed
SMPS connector	Power from the Switched Mode Power Supply (24-pin ATX main + 4/8-pin CPU). SMPS converts AC mains to regulated +3.3 V, +5 V, +12 V DC
Expansion / front-panel headers, fan headers, jumpers	Case connections and configuration

The boot process (asked in CF TP-I §4 explicitly, and in Diploma)

1. POWER ON → SMPS stabilises → the Power Good signal releases the CPU from reset
2. The CPU begins executing at a fixed address in the BIOS/UEFI ROM
3. POST (Power On Self Test): tests CPU, RAM, keyboard, video, storage
– errors reported by BEEP CODES (video not yet available)
4. BIOS reads its configuration from CMOS (boot order, time, settings)
5. BIOS loads the MASTER BOOT RECORD (MBR) – the first 512 bytes of the boot device – into memory at 0x7C00 and jumps to it
(or, with UEFI + GPT, loads the EFI bootloader from the EFI System Partition)
6. The MBR's boot code reads the partition table, finds the ACTIVE partition, and loads its VOLUME BOOT RECORD / bootloader (GRUB, Windows Boot Manager)
7. The bootloader loads the OS KERNEL into memory
8. The kernel initialises drivers, mounts the root file system, starts init/systemd or the Windows session manager
9. The login/user session begins

MBR structure (512 bytes): 446 bytes boot code + 64 bytes partition table (4 entries × 16 bytes) + 2 bytes signature **0x55AA**. **This exact breakdown is a standard forensic exam question.**

Likely exam questions — Storage and hardware

Q	Marks
Define track, sector, cylinder and cluster.	5
Compare SRAM and DRAM.	5
Compare SDRAM and DDR RAM.	5
Distinguish between PROM, EPROM and EEPROM.	5
Compare HDD and SSD under at least six headings.	5

A disk rotates at 7200 RPM with a 5 ms average seek time, 500 sectors per track and 512 B per sector. Compute the average access time for one sector and the data transfer rate. Comment on the result.	10
Describe the components of a motherboard and the function of each.	10
Explain the memory hierarchy from registers to secondary storage, giving typical capacity, access time and cost characteristics of each level, and explain how the principle of locality makes the hierarchy effective.	15
Describe the booting process of a computer from power-on to the login prompt, explaining the role of the SMPS, POST, BIOS/CMOS, the MBR and the bootloader.	15

MCQ traps — Storage and hardware

Trap	Truth
A cylinder is a track on one platter	A cylinder is the set of tracks at the same radius across all surfaces .
Average rotational latency = one full revolution	Half a revolution.
Transfer time dominates disk access	Seek time dominates by far.
Cache is bigger than main memory	It is much smaller — that is the entire premise.
EPROM is erased electrically	UV light . EEPROM is erased electrically.
ROM is volatile	Non-volatile . RAM is volatile.
Removing the CMOS battery erases the hard disk	It only clears BIOS settings and the BIOS password .
The MBR is 1024 bytes	512 bytes : 446 boot code + 64 partition table + 2 signature (0x55AA).
Blu-ray uses a red laser	Blue-violet, 405 nm . DVD uses red (650 nm); CD uses infrared (780 nm).
SSDs have seek time	No seek time — no moving parts.

PART H — LAST-WEEK REVISION SHEET

Formula sheet

ADDRESSING & MEMORY

Addressable memory = $2^{(\text{address bus width})}$
 Chips required = $(\text{total capacity} / \text{chip capacity}) \times (\text{word width} / \text{chip width})$
 Address lines per chip = $\log_2(\text{locations per chip})$

CACHE

Number of cache lines = cache size / block size
 Word/offset bits = $\log_2(\text{block size})$
 Direct: line bits = $\log_2(\text{lines})$; tag = addr - line - offset
 Associative: tag = addr - offset
 k-way set: sets = lines/k; set bits = $\log_2(\text{sets})$; tag = addr - set - offset
 $T_{\text{avg}} (\text{simultaneous}) = h \cdot T_c + (1-h) \cdot T_m$
 $T_{\text{avg}} (\text{hierarchical}) = T_c + (1-h) \cdot T_m$
 Two-level = $T_{L1} + (1-h_1) [T_{L2} + (1-h_2) \cdot T_m]$
 Global miss rate = $(1-h_1) (1-h_2)$

VIRTUAL MEMORY

Offset bits = $\log_2(\text{page size})$
 Pages = $2^{(\text{logical addr bits} - \text{offset bits})}$
 Frames = physical memory / page size
 Page table size = pages $\times$ entry size
 $EAT (TLB) = h(t_{TLB} + t_{mem}) + (1-h)(t_{TLB} + 2 \cdot t_{mem})$
 $EAT (\text{faults}) = (1-p) \cdot t_{mem} + p \cdot t_{\text{fault}}$

INTERLEAVING

n-way ideal speedup $\rightarrow n$
 Time for m consecutive words (n-way) = cycle_time + (m-1) $\cdot$ bus_time [m $\leq$ n]

PIPELINING

$T_{np} = n \cdot k \cdot t$ $T_p = (k + n - 1) \cdot t$
 Speedup $S = nk / (k+n-1)$ $S_{\text{max}} = k$
 Efficiency = $n / (k+n-1)$ Throughput = $n / [(k+n-1)t]$
 Clock period = slowest stage + latch delay
 Effective CPI = ideal CPI + stalls per instruction
 Amdahl: $S = 1 / [(1-f) + f/s]$

DISK

Capacity = surfaces $\times$ tracks/surface $\times$ sectors/track $\times$ bytes/sector
 Access time = seek + rotational latency + transfer
 Avg rotational latency = $0.5 \times 60/\text{RPM}$
 Transfer rate = (sectors/track $\times$ bytes/sector) / (60/RPM)

The 25 highest-value one-liners

1. CPU = ALU + Control Unit + Registers. Memory and I/O are outside it.
2. PC holds the **next** instruction's address; IR holds the **current** instruction.
3. Address bus is **unidirectional**; data bus is **bidirectional**.
4. n-bit address bus $\rightarrow 2^n$ addressable locations.
5. Von Neumann bottleneck = one shared bus for instructions and data.
6. Harvard = separate instruction and data memories/buses.
7. Immediate mode: **0** memory accesses. Direct: **1**. Indirect: **2**.
8. Stack machine = **0-address**, uses **postfix (RPN)**.
9. **Hardwired = fast, inflexible, RISC. Microprogrammed = slower, flexible, CISC.**

10. **Horizontal microcode = wide, unencoded, parallel, fast.**
11. Locality of reference: **temporal** (again soon), **spatial** (nearby soon).
12. **DRAM needs refresh** (capacitor); **SRAM does not** (flip-flop). SRAM = cache.
13. **DDR transfers on both clock edges**; SDRAM only on the rising edge.
14. **EPROM = UV erase**; **EEPROM = electrical erase**; **PROM = one-time**.
15. **Direct mapping needs no replacement algorithm.**
16. Address split: **tag | line-or-set | word-offset**. Offset bits = $\log_2(\text{block size})$.
17. **Write-through** keeps memory consistent; **write-back** uses a **dirty bit**.
18. **Low-order interleaving** puts consecutive addresses in **different** modules.
19. **TLB = cache of page table entries**, usually **fully associative**.
20. The page **offset passes through translation unchanged**.
21. **Paging** → **internal fragmentation**. **Segmentation** → **external fragmentation**.
22. **In DMA the data never passes through the CPU.**
23. **Cycle stealing does not interrupt the CPU** — it delays it by one bus cycle.
24. **Memory-mapped I/O uses LOAD/STORE**; **isolated I/O uses IN/OUT**.
25. **Pipelining improves throughput, not the latency of a single instruction.**
Max speedup = number of stages.

Exam tactics for this topic

- **Every numerical in this file is a 10- or 15-mark answer template.** Reproduce the steps with the question's numbers and you cannot lose structure marks.
- For cache questions, **always** write out the address partition diagram (**tag | index | offset**) with the bit counts labelled and verify they sum to the address width. That check alone earns marks and catches your own errors.
- When a hit-ratio question doesn't specify simultaneous vs hierarchical access, **state your assumption in one sentence and proceed**. Ambiguity punished is ambiguity unstated.
- Diagrams that earn marks in this paper: the functional block diagram, the memory hierarchy pyramid, the three cache mapping diagrams, the paging address-translation diagram, the DMA transfer diagram, and the pipeline space-time chart. **Practise drawing all six from memory in under two minutes each.**
- In the MCQ paper, the DMA / interrupt / addressing-mode questions are near-certain marks. The cache-numerical MCQs are worth the 60 seconds they take — do them, don't skip them.